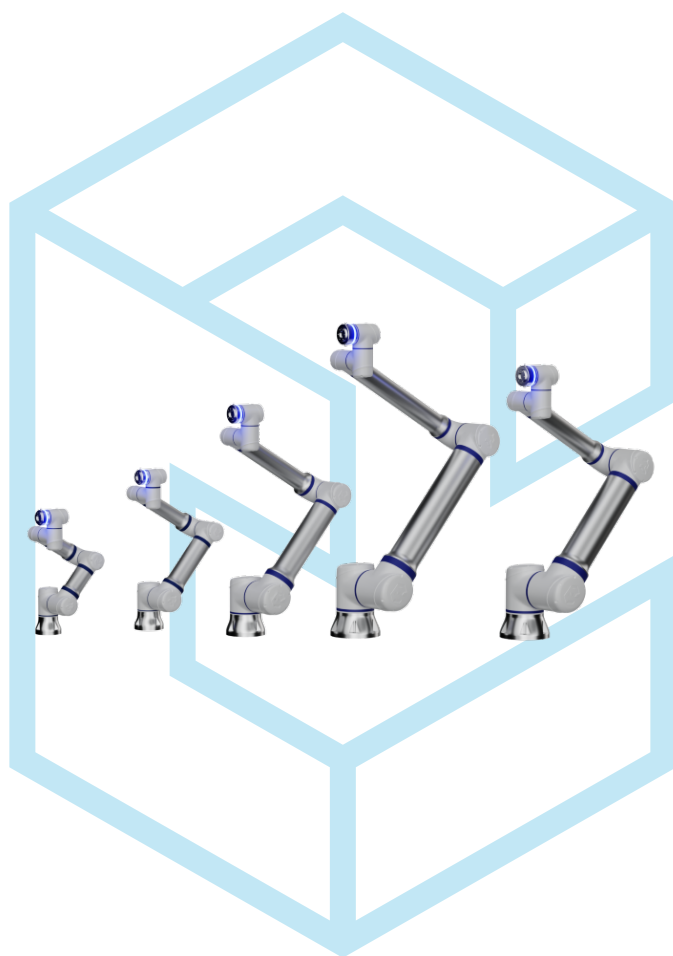


ELITE ROBOTS

Script Manual



Script Manual

Elite Intelligent Robot Co., Ltd.

2026-01-19

Version: Ver2.15.0

Please read this manual carefully before use

Please carefully check the version informations in this manual matches the corresponding software version of the system, to ensure consistency.

This user manual shall be periodically checked and revised, and the renewed contents will appear in the new version. The contents or information herein is subject to change without prior notice.

Elite Intelligent Robot Co., Ltd. shall assume no liability for any errors which will occur in the manual probably.

Elite Intelligent Robot Co., Ltd. shall assume no liability for the accident or indirect injury as a result of using this manual and the product mentioned herein.

Please read this manual before installing and using the product.

Please keep this manual so that you can read and use it for reference at any time.

The pictures in the specification shall be used for reference only. The goods received shall prevail.

Contents

1	EliteScript Programming Language	1
1.1	Introduction	1
1.2	Presentation Convention	1
1.3	EliteScript	2
1.3.1	Data Types	2
1.3.2	Variables	7
1.3.3	Conditional and Looping Statements	8
1.3.4	Function Definitions	12
1.3.5	Thread Usage	13
1.3.6	Thread Usage	14
2	Interpreter Mode	17
2.1	Instructions	17
2.1.1	Scripts Requirements	17
2.1.2	Requirements for Function Calling	17
2.2	Interpreter Mode Replies	18
2.2.1	Acknowledge (ack)	18
2.2.2	Skip Execution (discard)	18
2.3	Enter Interpreter Mode	19
2.4	End Interpreter Mode	20
2.5	Clear Injected Functions	21
2.6	State Commands for Interpreter Mode	21
2.7	Log Management for Interpreter Mode	23
3	Movement Command	25
3.1	MOVE-related command	25
3.1.1	movej command	25
3.1.2	movec command	26

3.1.3	move! command	27
3.1.4	get_actual_joint_positions command	28
3.1.5	get_actual_joint_speeds command	29
3.1.6	servoj command	29
3.1.7	get_actual_tcp_pose command	30
3.1.8	get_actual_tcp_speed command	31
3.1.9	get_controller_temperature command	31
3.1.10	get_joint_temperatures command	32
3.1.11	get_joint_torques command	32
3.1.12	get_target_joint_positions command	32
3.1.13	get_target_joint_speeds command	33
3.1.14	get_target_tcp_pose command	33
3.1.15	get_target_tcp_speed command	34
3.1.16	set_gravity command	34
3.1.17	set_payload command	34
3.1.18	get_target_payload_cog command	35
3.1.19	get_target_payload_mass command	35
3.1.20	set_tcp command	36
3.1.21	encoder_enable_set_tick_count command	36
3.1.22	encoder_get_tick_count command	36
3.1.23	encoder_set_tick_count command	37
3.1.24	encoder_unwind_delta_tick_count command	37
3.1.25	Configure the Incremental Encoder for the Conveyor(s)	38
3.1.26	speedj command	39
3.1.27	speedl command	40
3.1.28	stopj command	42
3.1.29	stopl command	42
3.1.30	track_conveyor_circular command	42
3.1.31	track_conveyor_linear command	43
3.1.32	stop_conveyor_tracking command	44

3.1.33	get_actual_tool_flange_pose command	44
3.1.34	get_target_waypoint command	45
3.1.35	is_within_safety_limits	45
3.1.36	get_tcp_offset command	47
3.1.37	load_micro_line_file command	47
3.1.38	micro_line_set_pcs command	48
3.1.39	get_micro_line_values_by_index command	48
3.1.40	moveml command	49
3.1.41	freedrive_mode command	49
3.1.42	end_freedrive_mode command	50
3.1.43	set_collision_detect_enabled command	50
3.1.44	is_collision_detect_enabled command	51
3.1.45	get_dyn_collision_force_min_ratio command	51
3.1.46	set_dyn_collision_force_min_ratio command	51
3.2	MATH-related command	52
3.2.1	get_inverse_kin command	52
3.2.2	get_inverse_kin_has_solution command	52
3.2.3	get_all_inverse_kin command	53
3.2.4	get_forward_kin command	54
3.2.5	bit_list_to_integer command	54
3.2.6	integer_to_bit_list command	55
3.2.7	get_list_length command	55
3.2.8	length command	56
3.2.9	acos command	56
3.2.10	asin command	56
3.2.11	atan command	57
3.2.12	atan2 command	57
3.2.13	cos command	58
3.2.14	sin command	58
3.2.15	tan command	58

3.2.16 d2r command	58
3.2.17 r2d command	59
3.2.18 ceil command	59
3.2.19 floor command	59
3.2.20 log command	60
3.2.21 sqrt command	60
3.2.22 pow command	60
3.2.23 norm command	61
3.2.24 normalize command	61
3.2.25 point_dist command	62
3.2.26 pose_add command	62
3.2.27 pose_dist command	63
3.2.28 pose_inv command	64
3.2.29 pose_sub command	64
3.2.30 pose_trans command	65
3.2.31 rotvec2rpy command	65
3.2.32 rpy2rotvec command	66
3.2.33 interpolate_pose command	66
3.2.34 random command	67
4 Force Command	69
4.1 force_mode command	69
4.2 force_mode_set_gain_scaling command	70
4.3 force_mode_set_damping command	71
4.4 zero_ftsensor command	72
4.5 force command	73
4.6 get_tcp_force command	73
4.7 end_force_mode command	74
4.8 Coordinate System Transformation	74
5 I/O Command	77

5.1	get_configurable_digital_in command	77
5.2	get_configurable_digital_out command	77
5.3	get_standard_analog_in command	77
5.4	get_standard_analog_out command	78
5.5	get_standard_digital_in command	78
5.6	get_standard_digital_out command	78
5.7	get_tool_analog_in command	79
5.8	get_tool_analog_out command	79
5.9	set_standard_analog_output_domain command	79
5.10	set_configurable_digital_out command	80
5.11	set_standard_analog_input_domain command	80
5.12	set_standard_analog_out command	81
5.13	set_standard_digital_out command	81
5.14	set_tool_analog_input_domain command	81
5.15	set_tool_analog_output_domain command	82
5.16	set_tool_digital command	82
5.17	get_tool_digital command	83
5.18	Set Tool Analog I/O Working Mode	83
5.19	Configure Tool Digital I/O	84
5.20	set_runstate_configurable_digital_output_to_value command	84
5.21	set_runstate_gp_boolean_output_to_value command	85
5.22	set_runstate_standard_analog_output_to_value command	86
5.23	set_runstate_tool_analog_output_to_value command	87
5.24	set_runstate_standard_digital_output_to_value command	88
5.25	set_runstate_tool_digital_output_to_value command	89
5.26	set_input_actions_to_default command	90
5.27	set_configurable_digital_input_action command	91
5.28	set_standard_digital_input_action command	92
5.29	set_tool_digital_input_action command	92
5.30	set_gp_boolean_input_action command	93

5.31 tool_serial_is_open command	94
5.32 tool_serial_config command	94
5.33 tool_serial_read command	96
5.34 tool_serial_write command	96
5.35 set_tool_voltage command	97
5.36 get_task_path command	97
5.37 read_input_boolean_register command	98
5.38 read_input_float_register command	98
5.39 read_input_integer_register command	98
5.40 read_output_boolean_register command	99
5.41 read_output_float_register command	99
5.42 read_output_integer_register command	99
5.43 write_output_boolean_register command	100
5.44 write_output_float_register command	100
5.45 write_output_integer_register command	101

6 Modbus Command 103

6.1 Modbus	103
6.1.1 modbus_add_signal command	103
6.1.2 modbus_delete_signal command	105
6.1.3 modbus_add_rw_signal	105
6.1.4 modbus_get_signal_status command	106
6.1.5 modbus_send_custom_command	107
6.1.6 modbus_set_output_register command	108
6.1.7 modbus_set_output_signal command	109
6.1.8 modbus_set_runstate_dependent_choice command	109
6.1.9 modbus_set_signal_update_frequency command	110
6.1.10 modbus_set_signal_watchdog	110
6.1.11 read_port_bit command	111
6.1.12 read_port_register command	111

6.1.13	write_port_bit command	112
6.1.14	write_port_register command	112
6.1.15	modbus_get_error	113
6.1.16	modbus_get_time_since_signal_invalid	114
6.1.17	modbus_reset_connection	115
7	Communication Command	117
7.1	SOCKET	117
7.1.1	socket_open command	117
7.1.2	socket_close command	117
7.1.3	socket_is_connected command	118
7.1.4	socket_get_var command	118
7.1.5	socket_read_ascii_float command	119
7.1.6	socket_read_binary_integer command	119
7.1.7	socket_read_byte_list command	120
7.1.8	socket_send_byte_list command	121
7.1.9	socket_read_string command	122
7.1.10	socket_send_byte command	123
7.1.11	socket_send_int command	124
7.1.12	socket_send_line command	125
7.1.13	socket_send_string command	125
7.1.14	socket_set_var command	126
7.2	RPC	126
7.2.1	rpc_factory command	126
8	Serial Communication	127
8.1	RS-485	127
8.1.1	tool_serial_mode command	127
8.1.2	masterboard_serial_config command	127
8.1.3	masterboard_serial_read command	128
8.1.4	masterboard_serial_write command	129

8.1.5	masterboard_serial_mode command	129
8.1.6	masterboard_serial_flush command	130
8.2	MODBUS-RTU	130
8.2.1	tool_modbus_read_bits command	130
8.2.2	tool_modbus_read_input_bits command	131
8.2.3	tool_modbus_read_registers command	131
8.2.4	tool_modbus_read_input_registers command	132
8.2.5	tool_modbus_write_single_bit command	132
8.2.6	tool_modbus_write_bits command	133
8.2.7	tool_modbus_write_single_register command	133
8.2.8	tool_modbus_write_registers command	134
8.2.9	masterboard_modbus_rtu_read_bits command	134
8.2.10	masterboard_modbus_rtu_read_input_bits command	135
8.2.11	masterboard_modbus_rtu_read_registers command	135
8.2.12	masterboard_modbus_rtu_read_input_registers command	136
8.2.13	masterboard_modbus_rtu_write_single_bit command	137
8.2.14	masterboard_modbus_rtu_write_bits command	137
8.2.15	masterboard_modbus_rtu_write_single_register command	138
8.2.16	masterboard_modbus_rtu_write_registers command	139

9 System Command 141

9.1	Thread	141
9.1.1	start_thread command	141
9.1.2	join_thread command	141
9.1.3	stop_thread command	142
9.2	Task execution	142
9.2.1	pause command	142
9.2.2	halt command	143
9.3	Prompts for a warning	143
9.3.1	textmsg command	143

9.3.2	popup command	143
9.4	Character	144
9.4.1	str_at command	144
9.4.2	str_cat command	144
9.4.3	str_empty command	145
9.4.4	str_find command	145
9.4.5	str_len command	146
9.4.6	str_sub command	146
9.4.7	to_num command	147
9.4.8	to_str command	148
9.5	Additional	149
9.5.1	sleep command	149
9.5.2	set_flag command	149
9.5.3	get_flag command	150
9.5.4	get_steptime command	150
9.5.5	controller_timestamp command	150
9.5.6	rtsi_set_watchdog command	151
9.5.7	powerdown command	151

Chapter 1 EliteScript Programming Language

1.1 Introduction

Elite Robots provides two programming methods: graphical programming and script programming. The graphical programming is completed by the Elite robots teach pendant interface, which can quickly realize the teaching and editing of points; the script programming is the underlying programming language, which can be programmed either through the graphical interface or through the text editor. Before the graphical program is sent to the controller for execution, it will be parsed into a scripting language, which will then be interpreted and executed by the controller; the graphical program can contain a scripting language, but not vice versa.

EliServer is the underlying control program in the controller. The user interface EliRobot, connects with EliServer through TCP/IP protocol, and sends the script program generated by the graphical interface to EliServer to run. The script program in the external device can also be connected to the EliServer through the TCP/IP interface to control the operation of the robot (need to enable the remote mode).

EliteScript is the Elite Robots scripting language, like any other programming language, EliteScript has variables, types, control flow statements, functions, etc. Additionally, EliteScript has many built-in variables and functions to monitor and control robot input/output and motion.

1.2 Presentation Convention

```
1 instruction(para1, para2=default_value2,..., <paraM=valuem, paraL=valueL  
  | paraK=valueK>, paraN= default_valueN)
```



Thereinto:

paraX=default_valueX indicates that this is an optional parameter. If it is not specified, the default value is used. If default_valueX is italic and quoted by single quotes, it represents a special meaning. For example, 'current_joint_positions' represents the current joint angle;

| Represents mutually exclusive parameters.

1.3 EliteScript

EliteScript is similar to the Python language, but also has some robot-related command and specific data types and usage methods. This section introduces commonly used variable types, syntax rules, customized syntax, keywords, threads and other functions in EliteScript. If the data types, syntax rules and other programming rules are not covered in this section, please refer to the default programming rules of the Python language and use them directly.

1.3.1 Data Types

EliteScript supports Number, String, List, Tuple, Dictionary generic Python data types, as well as custom data types such as Joint and Pose.

1.3.1.1 Number

Non-alterable data types

When its type is changed, it is assigned to a new object. When a variable is assigned a numeric value, the object is created and the reference to these objects can be removed by the del statement.

The number types supported by the script

Table 1-1. The number types supported by the script

Types	Remarks
int	Signed integer, such as 0x69, 10
boolean	boolean type, such as True, False
float	Float type, such as 70.2E-12
complex	Complex numbers, such as 4.53e-7j

1.3.1.2 String

Strings consist of numbers, letters, and underscores.

String interception

Strings in the script are intercepted from left to right: index range (0, length -1), right-toleft interception (-1, string beginning).

No single character in the script

Even if there is single character in the script, it is treated as Strings.

Script escape characters

Table 1-2. Script escape characters

Characters	Description
\	When it appears at the end of the line, it is displayed as a continuation character. When it appears in the line, it is used to "translate" special characters to express special meanings, as shown in the following options.
\\	Backslash symbol
\'	Single quotation marks
\"	Double quotation marks
\a	Bell
\b	Backspace
\e	Escape
\000	null
\n	Line breaks
\v	Vertical tab character
\t	Horizontal tab character
\r	Enter
\f	Page feeds
\oyy	Octal number, the character represented by yy, for example: \o12 represents a newline
\xyy	A hexadecimal number, the character represented by yy, for example: \x0a represents a newline
\other	Other characters are output in normal format

1.3.1.3 List

List data structures are used very frequently and support collection structures for numbers, characters, strings, and even lists.

Add or remove list elements

Reassign values directly to values taken out based on index values, or add them via the `append()` function. Delete list items by means of a `del` statement, for example: `dellist1[2]`.

The script operator for the list

Similar to working with strings.

Table 1-3. The script operator for the list

Method	Results	Description
<code>len([1,2,3])</code>	3	Length
<code>[1,2,3] + [4,5,6]</code>	<code>[1,2,3,4,5,6]</code>	Combination
<code>['Hi!'] * 4</code>	<code>['Hi!','Hi!','Hi!','Hi!']</code>	Repetition
<code>3 in [1,2,3]</code>	True	Determines whether an element exists in the list
<code>for x in [1,2,3]: print x</code>	1 2 3	Iterate

List interception

Table 1-4. List interception

Method	Results	Description
<code>L[2]</code>	<code>'CS612'</code>	Reads the third element in the list
<code>L[-2]</code>	<code>'CS66'</code>	Reads the second last element in the list
<code>L[1:]</code>	<code>['CS66','CS612']</code>	Start with the second element to intercept the list

Note: List L `['CS63','CS66','CS612']`

Functions and methods of lists in scripts

The script contains the following functions:

Table 1-5. Functions of lists in scripts

Method name	Description
<code>len(list)</code>	The number of list elements
<code>max(list)</code>	Returns the maximum value of a list element
<code>min(list)</code>	Returns the minimum value of a list element
<code>list(seq)</code>	Converts tuples to lists

The script contains the following methods:

Table 1-6. The method of the list in the script

Method name	Description
<code>list.append(obj)</code>	Add a new object at the end of the list
<code>list.count(obj)</code>	Counts the number of times an element appears in the list
<code>list.extend(seq)</code>	Append multiple values from another sequence at the end of the list at once (expand the original list with a new list)
<code>list.index(obj)</code>	Find the index position of the first match for a value from the list
<code>list.insert(index,obj)</code>	Inserts an object into the list
<code>list.pop(obj=list[-1])</code>	Removes an element from the list (the default last element) and returns the value of that element
<code>list.remove(obj)</code>	Removes the first occurrence of a value in the list
<code>list.reverse()</code>	An element in the reverse list
<code>list.sort([func])</code>	Sorts the original list

1.3.1.4 Tuple

Differences from lists

Similar to lists, but lists are identified by `[ ]`, tuples are identified by `()`, and list elements can be reassigned, but tuple elements cannot.

The creation of tuples

Create an empty tuple: `tuple()`.

To create a tuple with only one element: `tuple(a,)`, a comma must append to the element.

Access to the element.

Although it is created with `()` inclusion, but similar to the lists, the individual elements of the tuples can be accessed by `[index number]`.

Delete the tuple

Individual elements in a tuple cannot be deleted, but tuples can be deleted entirely by the `del` statement.

Tuple operator (same list)

Arbitrary unsigned objects, separated by commas, default to tuples (no closing separator)

Tuple built-in functions

Table 1-7. Tuple built-in functions

Methods	Description
<code>len(tuple)</code>	Calculates the number of tuple elements.
<code>max(tuple)</code>	Returns the maximum value of an element in a tuple.
<code>min(tuple)</code>	Returns the minimum value of an element in a tuple.
<code>tuple(seq)</code>	Converts a list to a tuple.

1.3.1.5 Dictionary

Differences from lists

Lists are ordered collections of objects, and dictionaries are unordered object combinations. Elements in the dictionary are fetched by Key, while elements in a list are fetched by displacement.

Dictionary definitions

Here are two ways to define a dictionary, both are similar to those for the list.

```

1 dict={}
2 dict['one']="This is one"
3 dict[2]="This is two"
4 tinydict={'name':'john','code':6734,'dept':'sales'}

```



Conversion of data types

Table 1-8. Conversion of data types

Methods	Description
<code>int(x [,base])</code>	Converts x to an integer
<code>float(x)</code>	Converts x to a floating-point number
<code>complex(real [,imag])</code>	Create a complex number
<code>str(x)</code>	Converts the object x to a string
<code>repr(x)</code>	Converts the object x to an expression string
<code>eval(str)</code>	Used to calculate a valid Python expression in a string and returns an object
<code>tuple(s)</code>	Converts the sequence s to a tuple
<code>list(s)</code>	Converts the sequence s to a list
<code>set(s)</code>	Converts to a mutable collection
<code>dict(d)</code>	Create a dictionary. d must be a sequence (key, value) tuple.
<code>frozenset(s)</code>	Converts to an immutable collection
<code>chr(x)</code>	Converts an integer to a character
<code>unichr(x)</code>	Converts an integer to Unicode characters
<code>ord(x)</code>	Converts a character to its integer value
<code>hex(x)</code>	Converts an integer to a hexadecimal string
<code>oct(x)</code>	Converts an integer to an octal string

1.3.2 Variables

The EliteScript scripting language divides variables into two types: local variables and global variables according to the scope of the variables. The global variables with

the global keyword defined in the script will appear on the variable monitoring page of EliRobot.

1.3.2.1 Global variables

When defining global variables, define them with the global keyword.

1.3.2.2 Local variables

Local variable is defined within a function and can only be used within the corresponding block of code.

```
1 # Define global variables
2 global total
3 total=0
4
5
6 # Write a description of the function
7 def sum(arg1, arg2): # Returns the sum of 2 parameters
8     total=arg1 + arg2 # total here are local variables
9     print("Inside the function are local variables : ", total)
10    return(total)
11
12
13 # Call the sum function
14 sum(10, 20);
15 print("Outside the function are global variables: ", total)
```

In the above example, total is a local variable inside the function, while external total is a global variable, Changing the local variable does not change the value of the global variable, so the first print result is 30, while the second is 0.

1.3.3 Conditional and Looping Statements

1.3.3.1 Conditional statements

Scripting languages do not support switch statements, so when judging that the result corresponds to multiple execution methods, it can only be implemented with elif or multi-level nested if statements.

```

1 num=5
2 if num==3:      # Determine the value of num
3     print('boss')
4 elif num==2:
5     print('user')
6 elif num==1:
7     print('worker')

```



1.3.3.2 Looping statements

There are no do while loops in the scripting language, and the supported loop statements are as follows:

Table 1-9. Looping statements

Loops	Description
while loops	Executes the loop body when the given judgment condition is true, otherwise exits the loop body.
for loops	Repeat the execution of the statement
Nested loops	For loops can be nested in the while loop body (for loops can also be nested in for loops)

Loop control statements:

Table 1-10. Loop control statements:

Control statements	Description
break statement	Terminates the loop during execution of the statement block and exits the entire loop
continue statement	Terminate the current loop during statement block execution, jump out of the loop, and execute the next loop.
pass statement	Pass is an empty statement to maintain the integrity of the program structure.

The role of the pass statement in functions

When writing a program, if the execution statement part of the idea has not been completed, then the pass statement can be used to occupy the place. The pass statement can also be used as a markup for completing later. For example:

```
1 def test():
2     pass
3 end
```



Define a function test, but the function body part is not yet complete, and it cannot be empty and not write content, so pass can be used to occupying a position.

The role of the pass statement in the loop

Pass is also commonly used to write an empty body for compound statements, for example: an infinity loop of a while statement that doesn't require any action at each iteration can be like:

```
1 while True:
2     pass
3 end
```



The above is just an example, in reality it is best not to write such code, because the execution block is pass, that is, empty and does nothing, then python will enter an endless loop.

Summary of pass statement usage

1. Empty statements, do nothing;
2. It is used in special cases to ensure the integrity of format or semantics.

While loops(The else statement can be used with a loop)

\# continue and break usage:

```
1 i=1
2 while i<10:
3     i+=1
4     if i%2>0: # Skip output if non-double
5         continue
6     print(i) # Outputs double numbers 2, 4, 6, 8, 10
7 end
8
9
10 i=1
11 while 1: # A cyclic condition of 1 must be true
12     print(i) # Outputs 1~10
```




```
13     i+=1
14     if i>10: # The loop is bounced out while i is greater than 10
15         break
16 end
```

Use the else statement in the loop, that is, when the condition is not met, the loop is terminated and the else statement is executed.

```
1 count=0
2 while count <5:
3     print(count," is less than 5")
4     count=count +1
5 else:
6     print(count," is not less than 5")
```

for loops (The else statement can be used with a loop)

Iterates through direct fetching or through sequence indexes.

Value iteration:

```
1 for letter in 'Python': # Output the characters in the string one by
2 one, and each output character is temporarily defined as letter (the
3 variable name can be replaced with any other name)
4     print('The current letter:', letter)
5
6
7 fruits=['banana', 'apple', 'mango']
8 for fruit in fruits: # Output the elements in the string one by one,
9 and each output element is temporarily defined as fruit (the variable
10 name can be changed to any other name)
11     print('Current fruit:', fruit)
12 print("Good bye!")
```

Index iteration:

```
1 fruits=['banana','apple','mango']
2 # The length of the list is obtained by the len() function, and a
3 sequence of indexes with values up to -1 in length is obtained by the
4     range() function
5 for index in range(len(fruits)):
6     print('Current fruit:',fruits[index])
7     print("Good bye!")
```

1.3.4 Function Definitions

1.3.4.1 Function definitions

```
1 # Standard function definitions
2 def mult(a, b):
3     return a*b
4 end
5
6
7 # Function definition with default parameters
8 def add(a=3, b=3):
9     return a+b
10 end
11
12
13 # Note: End is a special keyword for EliteScript that represents the
14 # end of the block, but is not necessary for using functions. The
15 # function can still be defined with standard Python syntax.
```



1.3.4.2 The function calls the method

```
1 ret_mult=mult(2, 4)
2
3
4 # Function definitions with default parameters can be called without
  # passing parameters
5 ret_add=add()
```



1.3.4.3 Parameter usage rules when calling a function

Required parameters

Parameters must be passed in the same order as when the function is declared.

Keyword parameters

Passing parameters can be in a different order than function declarations because the script interpreter can match parameter values with parameter names.

Default parameters

If no value is assigned to a parameter when passing a parameter, the default value is maintained.



```
1 # Write a description of the function
2 def printinfo(name, age=35):
3     print("Name: ", name) # Print any incoming strings
4     print("Age ", age)
5     return;
6
7
8 # Call the printinfo function
9 printinfo(age=50, name="miki") # Using Keyword parameters here, so
10 parameters don't need to be same order as function declared.
11 printinfo(name="miki") # If there is no input value here, the output
    default value is 35.
```

Indefinite-length parameters (that is, parameter definitions that contain non-required parameters)

When number of parameters that will be passed in is unsured, it's an option to precede the "*" sign to the parameter names that can be entered without input. The "*" sign need to be preceded correspond to the input in order.



```
1 def printinfo(arg1,*vartuple):
2     print("Outputs: ",arg1) #"Prints any incoming parameters"
3     for var in vartuple:
4         print var
5     return
6
7
8 # Call the printinfo function
9 printinfo(10)
10 printinfo(70,60,50)
```

1.3.5 Thread Usage

The scripting language supports the users to create threads, which allow users to implement complex bot control logic.

1.3.5.1 Methods for creating and shutting down threads

```

1  # Threaded functions are defined in the same way as normal functions
2  def thread_add_func(a, b):
3      c=a + b
4      print(c)
5      return c
6  end
7
8
9  # Run the thread function, which is executed asynchronously. The return
10 value of the start_thread function is a thread handle that can be used
11 while the thread is stopped.
12 thread_handler=start_thread(thread_add_func, (1, 3))
13
14
15 # Stops the thread function from running, stop_thread the parameter
16 value of the function is the handle returned when the thread is
17 running.
18 stop_thread(thread_handler)

```

1.3.5.2 Considerations when using the threading feature

- Once the task stops running, all threads created are automatically terminated;
- Parameters can be passed when starting a thread function;
- For loop statements in sub-threads or main threads, please strictly avoid an infinite loop without time-consuming operations, which may affect the overall system performance. Typical time-consuming operations can be sleep(), sync(), and movement commands.

1.3.6 Thread Usage

The user can link the port 30001 and send the sec command to the control system for dynamically setting and getting the global variable data in the currently running task. In case of abnormal conditions of the task, the alarm will be triggered. The alarm will be described with the start of [SEC] Exception: and appear in the format of the warning in the log column for the easy view.

The sec script indicates the secondary task and must start with the characters "sec". The use of the sec script is the same as that of the keyword "def" in the Python. You can find the scripting of the main task in the sec script. Like the def script, the sec script can be connected through TCP/IP and directly sent to the port 30001. You can also run the sec script when the main task script is running. The sec script has the same

format as that of the main task script. Note that the sec script does not support timeout parameters like sleep, socket and others which are related to ports.

All-important notes for the user:

- The running task will not be interrupted for any abnormal operations;
- Please do not run the sec script for a long time. If not, the unexpected outcomes may occur.

```
1 sec setDigitalOut():  
2   set_configurable_digital_out(1, True)  
3 end
```



Chapter 2 Interpreter Mode

Interpreter Mode overcomes the restriction on dynamic script injection by enabling the real-time injection of Python script statements during task runtime via port 30020. Once the mode is activated, users can instantly send valid Python statements to the controller queue. These statements are executed in queue order (max 8500 entries) without interrupting the running main program.

Key Features:

- Real-time script injection: dynamically insert commands during runtime;
- Function injection: inject callable functions across scripts

2.1 Instructions

2.1.1 Scripts Requirements

1. The statement is required to be a single-line, and end with `\n`, for instance:

```
1 set_tcp([0,0,0,0,0,0])\n
```



2. To inject functions or more complex control scripts, send them in a single line separated by semicolons. The overall spacing alignment follows Python formatting requirements. Here is the format.

```
1 def test_func():; textmsg("interpreter test"); popup("interpreter\npopup");end;
```



2.1.2 Requirements for Function Calling

1. In Interpreter Mode, defining new global variables is prohibited but those in main programs can be used and modified.
2. The defined function `test_func` that is sent to the controller can be called by others, but it will be cleared after the function `clear_interpreter` is executed. The mode does not support the calling of the function `start_thread` currently. If this function is included in those scripts to be sent, it will be discarded.

3. When Interpreter Mode runs time-consuming scripts that are not related to movement (such as the sleep command), it will exit automatically until the sleep command has finished executing. Even if the `end_interpreter()` command is called midway, time-consuming scripts will not be forcefully stopped running.

NOTICE



In Interpreter Mode, operations for controlling task execution status (such as pause/resume/stop, etc.) take effect synchronously.

2.2 Interpreter Mode Replies

2.2.1 Acknowledge (ack)

When Interpreter Mode receives executable script statements, if they have been compiled successfully and added to the queue, the following format of information will be returned to notify the client:

ack: <id>: <statements>

Among these, 'id' is the unique ID assigned to the accepted statement. Subsequent status command return values will use this 'id' information.

2.2.2 Skip Execution (discard)

If the received script is not properly pushed into the interpreter queue, the interpreter will skip execution and return a discard message along with the reason, in the following format:

discard: <reason>: <statement>

Among the message, 'reason' is the reason, and 'statement' is the returned abandonment information.

Four common internal triggers may cause a discard (skipping execution), as detailed in **Table 2-1**. The specific triggers can be determined based on the information in the returned message.

Table 2-1. Triggers of Skipping Execution

Type	Reasons	Returned Message
Compilation error	This usually indicates an error in the script's format. Please check if the script sent is a valid Python statement	discard: Compile error:<trace info> : <statement>
Task status error	The current task is not running.	discard: Task is in an invalid state: <statement>
Script queue length exceeded	The current script queue length exceeds the maximum value (8500)	discard: Too many interpreted messages: <statement>
Queue was cleared before interpreter execution	Because the clearQueueOnEnter parameter is set to True, scripts sent to the interpreter queue before Interpreter Mode is active are discarded upon entry (i.e., when calling the interpreter_mode script).	discard: Cleaned up before interpretation: <statement>

2.3 Enter Interpreter Mode

```
interpreter_mode(clearQueueOnEnter = True, clearOnEnd = True)
```

Function: This command is used to enter the interpreter mode. This is a blocking function and can only be called from the main thread but not nested.

Parameter: `clearQueueOnEnter`:

True = When interpreter mode is activated, clear the script queue received before activation.

False = When interpreter mode is activated, the existing queue is not be cleared and the sytem immediately starts executing any statements already in the queue.

`clearOnEnd`:

True = Clears all functions injected into the interpreter mode (by executing the `clear_interpreter` command) when exiting the interpreter mode.

False = When exiting Interpreter Mode, keep all injected statements. The injected statements remain valid as part of the program until the program finishes execution. When `interpreter_mode()` is called to enter interpreter mode subsequently , these statements can still be called or accessed.

Return: /

Example:

```
interpreter_mode(): to activate the interpreter mode in the
default way.
interpreter_mode(clearQueueOnEnter = True, clearOnEnd = True):
to activate the interpreter mode (clearing the queue upon
entry and the injected upon exit)
interpreter_mode(clearQueueOnEnter = False): to activate the
interpreter mode (does not clear the queue upon entry).
interpreter_mode(clearOnEnd = False): to activate the
interpreter mode (keeping all injected statements upon exiting
)
```

2.4 End Interpreter Mode

```
end_interpreter()
```

Function: This command is used to disable the interpreter mode and unblock the `interpreter_mode` function.

This function can be called in two ways. First, call it in a sub-thread of a task. Second, wait for calling it in the script queue after it is sent to interpreter mode via port 30020.

At the end, all interpreted statements will be cleared, but the following changes will be retained: robot status, modifications to local variables in the enclosing scope, and changes to global variables. The interpreter thread will be idle after this operation.

Parameter: /

Return: /

Example: `end_interpreter()`

2.5 Clear Injected Functions

```
clear_interpreter()
```

Function: This command is used to clear functions injected into the interpreter. Once a function has been cleared from injection, it can no longer be called within the interpreter.

After the `clear_interpreter()` function in the queue has been executed an ack operation, the system will stop executing an ack operation for new script that is waiting at this time. The script that is waiting will be acknowledged (ack) only after this command is executed.

Parameter: /

Return: /

Example: `clear_interpreter()`

2.6 State Commands for Interpreter Mode

The following commands will be executed immediately upon receipt and will not be queued for execution.

Table 2-2. State Commands

Command	Function	Returned Message
skipbuffer	to skip all received but non-executed script commands. Note that the command currently being executed will continue to run until it finishes.	-
statelastexecuted	to return the ID of the last executed command (or currently being executed)	state: <id>: statelastexecuted
statelastinterpreted	to return the ID of the most recently parsed command	state: <id>: statelastinterpreted
statelastcleared	to return the last cleared command (referring to commands cleared when clear_interpreter is called or Interpreter Mode is disabled).	state: <id>: statelastinterpreted
stateunexecuted	to return the unexecuted commands in the current queue (including those waiting to be skipped).	state: <#unexecuted>: stateunexecuted
Abort	to terminate the currently running motion command in the program (only supports terminating movej and movel commands). Its returned value can be ignored even if the motion command is not triggered through Interpreter Mode.	

2.7 Log Management for Interpreter Mode

Interpreter Mode record its data with a separate log file. When a safety alarm is triggered or a flight record is packaged, the system exports this log to the folder within the flight record in EliServer/controller/eli_log folder .

The default path for storing log data is as follows:



```
1  "/tmp/log/EliServer/interpreter.log" # Interpreter script log, used to
   record received interpreter scripts, discard information, etc.
2  "/tmp/log/EliServer/interpreter.0.log" # history log of Interpreter
   script log
3  "/tmp/log/EliServer/interpreter.execute.log" # Interpreter execution
   log, used to record the interpreter's execution process, such as the
   script being executed and its corresponding ID information.
4  "/tmp/log/EliServer/interpreter.execute.0.log" # history log of
   Interpreter execution log
```


Chapter 3 Movement Command

This chapter describes the function, parameters, and other related information for all movement commands.

3.1 MOVE-related command

This section mainly introduces MOVE related commands.

NOTICE



- In all script commands for the movement of CS series robots, the robot speed percentage has certain impacts on the controlling speed.
- The commands of joint movement and linear movement can be used only when the robot is stopped or in the transitional stage between movej and movel.

3.1.1 movej command

```
movej(q,a=1.396263,v=1.047197,t=0,r=0)
```

- Function:** This command is used for joint movement, that is, to move the robot to a target position (q).
- Parameter:**
- q: joint positions of the target point (the user can use the inverse kinematics function to convert Cartesian coordinates into the joint coordinates and then input the result), the format is [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], list type data. The unit is rad;
 - a: joint acceleration (unit: rad/s^2), float type data, with a value range greater than 0. It defaults to 1.396263 rad/s^2 (equivalent to $80^\circ/\text{s}^2$). If its value is set to be too large, it will be constrained according to the maximum allowable acceleration of each joint. The maximum value varies with axis type and payload.
 - v: joint velocity (unit: rad/s), float type data, larger than 0. The default value is 1.047197 rad/s (equivalent to $60^\circ/\text{s}$). If its value is set to be too large, it will be restricted according to the maximum moving capability of each joint. Similarly, its maximum value depends on the type of axis and payload.
 - t: motion duration (unit: s), float type data. It is defaults to 0s; the system will plan the trajectory based on the given parameters (a and v) at this time. If it is larger than 0s, then a and v will be ignored and the system will carry out motion planning as required.
 - r: blending radius (unit: m), float type data. The default value is 0 m, and its maximum is influenced by the distance between adjacent waypoints.

Return: None

Example: `movej([0.57636, -1.01469, -2.04816, -1.29723, 1.5708, 0], a=1.396263, v=1.047197, t=0, r=0)`

Example command parameters:

q=[0.57636, -1.01469, -2.04816, -1.29723, 1.5708, 0] -> the joint position of the target point is set as: base=0.57636rad, shoulder=-1.01469rad, elbow=2.04816rad, wrist1=-1.29723rad, wrist2=1.5708rad, wrist3=0rad

a=1.396263 -> the joint acceleration is 1.396263 rad/s^2

v=1.047197 -> the joint velocity is 1.047197 rad/s

t=0 -> the motion duration is not specified

r=0 -> the blending radius is 0m

3.1.2 movec command

```
movec(pose_via, pose_to, a=1.2, v=0.25, r=0, mode=0)
```


- Function:** This command is used for arc trajectory motion. The robot will start its motion from the current position to the target position `pose_to` passing through an intermediate point `pose_via`. The robot will accelerate to a tool speed `v`, then moves constantly at speed `v`.
- Parameter:**
- `pose_via`: Intermediate point (`Pose_via` can also be specified as joint positions, the user can use the forward kinematics function to convert the joint space coordinates into Cartesian coordinates and then input the result), the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` unit is `m`, `Rx, Ry, Rz` unit is `rad`;
 - `pose_to`: target pose (the user can use the forward kinematics function to convert the joint space coordinates into Cartesian coordinates and then input the result), the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` unit is `m`, `Rx, Ry, Rz` unit is `rad`;
 - `a`: tool acceleration (unit: m/s^2), float type data, with a value range greater than 0. It defaults to 1.2 m/s^2 . If its value is set to be too large, it will be constrained according to the maximum allowable acceleration. The maximum value varies with payload.
 - `v`: tool speed (unit: `m/s`), float type data, larger than 0. The default value is `0.25m/s`. If its value is set to be too large, it will be restricted according to the maximum moving capability. Similarly, its maximum value depends on payload.
 - `r`: blending radius (unit: `m`), float type data. The default value is `0 m`, and its maximum is influenced by the distance between adjacent way-points.
 - `mode`: arc mode (0 is fixed mode, other values are at unconstrained mode, default is 0 if not written), integer type data.

Return: None

Example:

```
movec([0.41951, -0.16, 0.25745, -3.11757, -0, -1.5708],  
[0.41951, -0.00562, 0.25745, -3.11757, 0, -1.5708], a=1.2, v=0.25, r=0,  
mode=0)
```

3.1.3 movel command

```
movel(p, a=1.2, v=0.25, t=0, r=0)
```

Function: This command is used for linear movement. The robot can move to a target position (p) linearly through this command. When using this command, the robot must be in a standstill, or transition state of movej and movel.

Parameter: p: target pose (the user can use the forward kinematics function to convert the joint space coordinates into Cartesian coordinates and then input the result), the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z unit is m, Rx, Ry, Rz unit is rad;

a: tool acceleration (unit: m/s^2), float type data, with a value range greater than 0. It defaults to $1.2 m/s^2$. If its value is set to be too large, it will be constrained according to the maximum allowable acceleration. The maximum value varies with payload.

v: tool speed (unit: m/s), float type data, larger than 0. The default value is 0.25m/s. If its value is set to be too large, it will be restricted according to the maximum moving capability. Similarly, its maximum value depends on payload.

t: motion duration (unit: s), float type data. It defaults to 0s; the system will plan the trajectory based on the given parameters (a and v) at this time.

r: blending radius (unit: m), float type data. The default value is 0 m, and its maximum is influenced by the distance between adjacent waypoints.

Return: None

Example: `movel([0.41951,-0.16,0.25745,-3.11757,0,-1.5708],a=1.2,v=0.25,t=0,r=0)`

Example Command Parameters:

p=[0.41951,-0.16,0.25745,-3.11757,0,-1.5708] -> the target pose is set as: x=0.41951m, y=-0.16m, z=0.25745m, Rx=-3.11757rad, Ry=0rad, Rz=-1.5708rad

a=1.2 -> the tool acceleration is $1.2m/s^2$

v=0.25 -> the tool speed is 0.25m/s

t=0 -> the motion duration is unspecified

r=0 -> the blending radius is 0m

3.1.4 get_actual_joint_positions command

```
get_actual_joint_positions()
```

Function: This command obtains the actual joint positions of each joint. The position is returned in radians as a vector of length 6.

Parameter: None

Return: List type data: [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], the current actual joint angle position in rad.

Example:

```
global j
j=get_actual_joint_positions()
print(j)
Return value: The current joint angle position
```

Note: The output may differ from the output of `get_target_joint_positions()`, especially when robot is accelerating or carrying heavy loads.

3.1.5 get_actual_joint_speeds command

```
get_actual_joint_speeds()
```

Function: This command obtains the actual joint velocity of each joint. The velocity is returned in rad/s as a vector of length 6.

Parameter: None

Return: List type data: [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], the current actual joint speed in rad/s.

Example:

```
global b
b=get_actual_joint_speeds()
print(b)
Return value: The current joint speed
```

Note: The output may differ from the output of `get_target_joint_speeds()`, when robot is accelerating or carrying heavy loads.

3.1.6 servoj command

```
servoj (q, t=0.004, lookahead_time=0.1, gain=300)
```

Function: This command is used for real-time control of the robot's joint positions. Within the look-ahead time, the system processes the received joint angles at the specified time interval, applies mean filtering, and then performs spline fitting on the filtered data to generate smooth joint positions for real-time control.

Parameter: q: target joint position (unit: radians), list type data;
t: the control duration of the current command (unit: s). Default value is 0.004. Execution will block for the duration, float type data (optional parameter);
lookahead_time: motion look-ahead time (unit: s), used to calculate the look-ahead position. Smaller values provide faster tracking response, while larger values reduce overshoot during tracking. This parameter is 0.1 by default and its range is [0.03, 0.2], float type data (optional parameter).
gain: control gain which determines the speed adjustment strength during tracking. The higher the value, the faster the response. This parameter is 300 by default and its range is [100, 2000], integer type data (optional parameter).

Return: None

Example:

```
servoj([0.0,1.57,-1.57,0,0,3.14], 0.004, 0.1, 300)
q = [0.0,1.57,-1.57,0,0,3.14] -> target joint positions
t = 0.004 -> control duration (s), current command blocks for 0.004s
lookahead_time = 0.1 -> look-ahead time (s)
gain = 300 -> control gain
```

NOTICE



- A high gain or a low lookahead_time may cause the robot to vibrate and reduce motion smoothness, especially when the target joint position is noisy or the update frequency is low.
- It is recommended to use a smaller value for the t parameter (such as the default 0.004 seconds) whenever the application scenario allows.

3.1.7 get_actual_tcp_pose command

```
get_actual_tcp_pose()
```

Function: This command is used to obtain the current TCP pose. Return 6D pose
- The 6D pose represents the tool position and orientation specified in the base frame. The calculation of this pose is based on the actual robot encoder readings.

Parameter: None

Return: The current actual TCP pose is in the format of [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
global b
b=get_actual_tcp_pose()
print(b)
```

Return value: The current tool TCP pose

3.1.8 get_actual_tcp_speed command

```
get_actual_tcp_speed()
```

Function: This command is used to obtain the current TCP speed. Return the speed list with the length of 6. The first three values are velocity component in the direction of x, y and z axis of TCP. The last three values indicate the direction of the rotation axis in the current TCP pose.

Parameter: None

Return: The current actual TCP speed vector, the format is [Vx, Vy, Vz, RVx, RVy, RVz], list type data, where Vx, Vy, Vz indicates the velocity component in the direction of x, y and z axis of TCP, units are m/s, RVx, RVy, RVz indicates the direction of the rotation axis in the current TCP pose (based on the base coordinate system), the length of the vectors indicates the rotating speed on the rotation axis, units are rad/s.

Example:

```
get_actual_tcp_speed()
```

Return value: The current actual TCP speed vector

3.1.9 get_controller_temperature command

```
get_controller_temperature()
```

Function: This command is used to obtain the control box motherboard temperature.

Parameter: None

Return: Float type data, control box motherboard temperature, unit: Celsius.

Example: `get_controller_temperature()`
Return value: Control box motherboard temperature, unit: Celsius

3.1.10 get_joint_temperatures command

```
get_joint_temperatures()
```

Function: This command obtains the torque of each joint in Nm as a vector of length 6.

Parameter: None

Return: List type data: [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], the temperature of all joints, unit: Celsius.

Example: `get_joint_temperatures()`
Return value: Temperature of all joints, in Celsius

3.1.11 get_joint_torques command

```
get_joint_torques()
```

Function: This command is used to obtain the torques of all joints, in units: Nm; Joint torque - through the required torque correction to make the robot itself move (gravity, friction, etc.), returned in a vector of length 6.

Parameter: None

Return: List type data, joint torque vector.

Example: `global l`
`l=get_joint_torques()`
`print(l)`
Return value: Joint torque of 1 to 6 axes

3.1.12 get_target_joint_positions command

```
get_target_joint_positions()
```

- Function:** This command obtains the target joint position of each joint. The position is returned in radians as a vector of length 6.
- Parameter:** None
- Return:** List type data: [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], the current target joint angle position in rad.
- Example:** `get_target_joint_positions()`
Return value: Joint target joint angle positions from 1 to 6 axes
- Note:** The output may differ from the output of `get_actual_joint_positions()`, when robot is accelerating heavy loads.

3.1.13 get_target_joint_speeds command

```
get_target_joint_speeds()
```

- Function:** This command is used to get the target joint velocities of all joints. The target joint velocity is in rad/s and returned in a vector of length 6.
- Parameter:** None
- Return:** List type data: [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], the target joint speed in rad/s.
- Example:** `get_target_joint_speeds()`
Return value: The target joint speed

3.1.14 get_target_tcp_pose command

```
get_target_tcp_pose()
```

- Function:** This command is used to get the current target tool pose. Returns a 6D pose, which represents the tool position and orientation specified in the base frame. The calculation of this pose is based on the current target joint positions.
- Parameter:** None
- Return:** The current target TCP vector: [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.
- Example:** `get_target_tcp_pose()`
Return value: The current target TCP vector

3.1.15 get_target_tcp_speed command

```
get_target_tcp_speed()
```

Function: This command is used to obtain the target TCP speed. Return the speed list with the length of 6. The first three values are velocity component in the direction of x, y and z axis of TCP. The last three values indicate the direction of the rotation axis in the current TCP pose

Parameter: None

Return: The target TCP speed vector, the format is [Vx, Vy, Vz, RVx, RVy, RVz], list type data, where Vx, Vy, Vz indicates velocity component in the direction of x, y and z axis of TCP, units are m/s, RVx, RVy, RVz indicates the direction of the rotation axis in the current TCP pose (based on the base coordinate system), the length of the vectors indicates the rotating speed on the rotation axis, units are rad/s.

Example:

```
get_target_tcp_speed()  
Return value: The target TCP speed vector
```

3.1.16 set_gravity command

```
set_gravity(d)
```

Function: This command is used to set the direction of the acceleration experienced by the robot. When the robot mounting is fixed, this corresponds to the acceleration away from the earth's center.

Parameter: d: 3D vector, describing the direction of the gravity, relative to the base of the robot, list type data.

Return: None

Example:

```
set_gravity([0,9.82 * sin(theta),9.82 * cos(theta)])  
Note:Set the direction of the robot's gravitational acceleration  
to rotate theta radians around  
the X-axis of the robot's base frame.
```

3.1.17 set_payload command

```
set_payload(m, CoG, inertia=[0,0,0,0,0,0])
```


- Function:** This command is used to set the mass, center of gravity and moment of inertia of the robot payload; When the payload of the robot changes significantly (such as picking up or dropping a heavier workpiece), the function needs to be called to set the new payload information.
- Parameter:** m: the mass of the payload, float type data: the range is not less than 0, not greater than the maximum nominal load of the current model robot, unit: kg;
CoG: The coordinates of the center of gravity of the payload (relative to the flange frame), list type data: [CoGx, CoGy, CoGz], unit: m;
inertia: moment of inertia of the payload, optional parameters, list type data: [Ixx, Iyy, Izz, Ixy, Ixz, Iyz], unit: kg/m^2 .
- Return:** None
- Example:** `set_payload(1,[0,0,0.12],[0,0,0,0,0,0])`

3.1.18 get_target_payload_cog command

```
get_target_payload_cog()
```

- Function:** This command obtains the center of the gravity coordinates of the current load that the robot is carrying. The units is m.
- Parameter:** None
- Return:** List type data [CoGx, CoGy, CoGz].
- Example:** `get_target_payload_cog()`
Return value: Center of gravity coordinates

3.1.19 get_target_payload_mass command

```
get_target_payload_mass()
```

- Function:** This command is used to obtain the mass of the robot's current payload.
- Parameter:** None
- Return:** Float type data, mass in kg.
- Example:** `get_target_payload_mass()`
Return value: The mass of the robot's current payload

3.1.20 set_tcp command

```
set_tcp(pose)
```

Function: This command is used to set the coordinates and pose of the center point of the robot tool.

Parameter: pose: A pose describing the transformation. [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: None

Example: `set_tcp([0,0,0.05,0,0.1,0.1])`

3.1.21 encoder_enable_set_tick_count command

```
encoder_enable_set_tick_count(encoder_index, range_id)
```

Function: This command is used to set up an encoder expecting to be updated with tick counts via the function.

Parameter: encoder_index: the ordinal number of the encoder, which must be 0 or 1, integer type data;
range_id: the ordinal number of the decoder: used to specify the range of the encoder, integer type data. The right range needs to be selected for the encoder,
0 is a 32-bit signed decoder with a range of [-2147483648 : 2147483647];
1 is an 8-bit unsigned decoder with a range of [0 : 255];
2 is a 16-bit unsigned decoder with a range of [0 : 65535];
3 is a 24-bit unsigned decoder with a range of [0 : 16777215];
4 is a 32-bit unsigned decoder with a range of [0 : 4294967295], integer type data.

Return: None

Example: `encoder_enable_set_tick_count(0,0)`

3.1.22 encoder_get_tick_count command

```
encoder_get_tick_count(encoder_index)
```

Function: This command is used to return the tick count of the selected encoder.

Parameter: encoder_index: the encoder ordinal number to be accessed, the ordinal value must be 0 or 1, integer type data.

Return: Float type data, the conveyor encoder tick count.

Example: encoder_get_tick_count(0)
Return value: The conveyor encoder tick count

3.1.23 encoder_set_tick_count command

```
encoder_set_tick_count(encoder_index, count)
```

Function: This command is used to tell the robot controller the tick count of the current encoder. The function can be used under absolute encoders(e.g. Modbus);
See also:2.1.20 encoder_enable_set_tick_count() command.

Parameter: encoder_index: encoder ordinal number, the ordinal value must be 0 or 1, integer type data;
count: The tick count to set. Must be within the range of the encoder, float type data.

Return: []

Example: encoder_set_tick_count(0,1234)

3.1.24 encoder_unwind_delta_tick_count command

```
encoder_unwind_delta_tick_count(encoder_index, delta_tick_count)
```

- Function:** This command is used to return refactored delta_tick_count if the encoder has counted out of bounds. If the encoder does not occur count out of bounds, the delta_tick_count will be returned directly without any modifications.
- Parameter:** encoder_index: the encoder ordinal number to be accessed, the ordinal value must be 0 or 1, integer type data;
delta_tick_count: the difference between the two tick counts that need to be reconstructed, float type data.
- Return:** Float type data, reconstructed delta_tick_count.
- Example:**
- ```
encoder_unwind_delta_tick_count(0, -64666)
illustrate:
Encoder assuming an ordinal number of 0, which has a range of [0 : 65535]
Assuming the first use of encoder_get_tick_count (0), the acquisition tick_count is 65530
Assuming that encoder_get_tick_count (0) is now used, the tick_count obtained is 864
At this point delta_tick_count=-64666 (864 - 65530=-64666)
Return value:870
```

### 3.1.25 Configure the Incremental Encoder for the Conveyor(s)

```
encoder_enable_pulse_decode(encoder_index, decoder_type, A, B)
```

- Function:** This command is used to configure the operating mode and high-speed I/O signals of a conveyor's incremental encoder.
- Parameter:** encoder\_index: the index of the target encoder (0 = Conveyor 0; 1 = Conveyor 1), integer type data;  
decoder\_type: to configure the mode of the incremental encoder. 1 = Quadrature Mode; 2 = Quadrature with Index Mode; 3 = Rising Edge Mode; 4 = Falling Edge Mode.  
A: to assign a high-speed I/O pin to channel A (16 = High Speed I/O 16; 17 = High Speed I/O 17).  
B: to assign a high-speed I/O pin to channel B If using the quadrature mode, channel B's I/O pin must be different from channel A's; otherwise, this parameter should be set as 0.
- Return:**
- Example:**
- ```
encoder_enable_pulse_decode(0, 1, 16, 17)
```

NOTICE



In Quadrature mode, channels A and B must be assigned different high-speed I/O signals.

3.1.26 speedj command

`speedj(qd, a, t)`

Function: This command is used to accelerate each joint to the target speed (qd) and then move continuously at this speed.

Parameter: qd: joint target angular speed (rad/s), a list containing 6 values corresponding to joints J1-J6. Positive values indicate positive rotation of the joint; negative values indicate rotation in the opposite direction.

a: joint acceleration (rad/s^2), a float value greater than 0, used to accelerate from the current speed to the target value (qd).

t: minimum function return time (s), an optional float value.

When t is not specified, the function returns only after the target speed is reached. The next command is executed afterwards. This is suitable for scenarios in which the target speed is required for subsequent commands (e.g., remote control operations).

When t is specified, if the target speed is reached early, the robot will keep moving at the target speed until the end of t seconds; if not reached within t seconds, the program will run the next command directly. In manual operation, specifying a suitable t provides a stable constant-speed motion, and also prevents deadlocks caused by unreachable target speeds.

Return: None

Example: `speedj([0.5,1,1,1,1,1],0.5,1)`

This example is used to accelerate each joint of the robot arm to the following target speeds at an acceleration of 0.5 rad/s^2 :

J1: 0.5 rad/s ; J2-J6: 1 rad/s .

If the target speed is reached within 1s, the robot maintains constant-speed motion until the end of 1s; if not reached within 1s, the program will directly proceed to the next command.

NOTICE

This command is affected by the "Speed Percentage". The target speed can be reached only when the speed percentage is set to 100%. For example, if the speed percentage is set to 50%, the actual running speed will be halved.

3.1.27 speedl command

```
speedl(xd, a, t, aRot='a')
```

Function: This command is used to cause the robot's tool end to accelerate in the robot base coordinate system along a specified direction to the target speeds (xd) and then move continuously at this speed.

Parameter: xd: speed at the tool end (m/s), a list of 6 values [Vx, Vy, Vz, RVx, RVy, RVz]. Positive values indicate motion in the positive coordinate directions, and negative values indicate motion in the negative directions. Vx, Vy, Vz define the linear speed (m/s) of the tool along the X, Y, Z axes of the base coordinate system; RVx, RVy, RVz specify the angular speed (rad/s) of the tool rotating around the X, Y, Z axes.

a: linear acceleration (m/s^2), a float value greater than 0, used to accelerate to the target value (xd).

t: minimum function return time (s), an optional float value.

When t is not specified, the function returns only after the target speed is reached, which is suitable for scenarios where the target speed is required for subsequent commands (e.g., remote control operations);

When t is specified, if the target speed is reached early, the robot will keep moving at the target speed until the end of t seconds; if not reached within t seconds, the program will proceed to the next command directly. In manual operation, specifying a suitable t provides a stable constant-speed motion, and prevents deadlocks caused by unreachable target speeds.

aRot: angular acceleration (rad/s^2), i.e., the acceleration of the TCP's angular speed. By default, it equals the linear acceleration a. This parameter is valid only when t is specified.

Return: None

Example: `speedl([-1.4E-4,-0.04,-0.02,0.0,0.0,-0.0],1.2,100.0)`

The example is used to accelerate the end of the tool to the following speeds at an acceleration of 1.2 m/s^2 :

moving in the negative direction along its own X axis at 0.00014 m/s , Y axis at 0.04 m/s , and Z axis at 0.02 m/s ; without rotary motion.

The program will wait 100s when it runs to this command. During this time, the robot arm will attempt to reach the target speed.

If the target speed is reached early, the constant motion will be maintained until the end of 100s; if not reached within 100s, the program will proceed to the next command directly.

NOTICE

This command is also affected by the "Speed Percentage" setting. The target speed can be reached only when the speed percentage is set to 100%. For example, if the speed percentage is set to 50%, the actual speed will be halved.

3.1.28 stopj command

```
stopj(a)
```

Function: This command is used to decelerate joint speeds to zero.

Parameter: a: Joint acceleration, unit is rad/s^2 , float type data.

Return: None

Example: `stopj(0.1)`

3.1.29 stopl command

```
stopl(a)
```

Function: This command is used to decelerate tool speed to zero.

Parameter: a: Joint acceleration, in m/s^2 ; float type data.

Return: None

Example: `stopl(0.5)`

3.1.30 track_conveyor_circular command

```
track_conveyor_circular(center, ticks_per_revolution, rotate_tool=False,  
    encoder_index=0)
```


Function: This command is used to enable the conveyor tracking function, allowing robot motions (movej(etc.)) to track a circular conveyor.

Parameter: center: pose vector, to determine the center position of the conveyor in the base frame of the robot, list type data;
ticks_per_revolution: the number of ticks of the encoder change when the conveyor is moved one turn, float type data;
rotate_tool: the tool should rotate with the conveyor or stay in the direction specified by the trajectory (movej(, etc.)). When no boolean value is specified, use the default boolean value as False, boolean type data (optional parameter);
encoder_index: the encoder serial number associated with the conveyor tracking, the ordinal number value must be 0 or 1. When no ordinal value is specified, use the default ordinal value of 0, integer type data (optional parameter).

Return: None

Example: `track_conveyor_circular([0.5,0.5,0,0,0,0],500.0,False)`
Parameters:
center=[0.5,0.5,0,0,0,0]
ticks_per_revolution=500.0
rotate_tool=False
encoder_index, =0The default value

3.1.31 track_conveyor_linear command

```
track_conveyor_linear(direction, ticks_per_meter, encoder_index=0)
```

- Function:** This command is used to enable the conveyor tracking function, allowing robot motions (movej(etc.)) to track a linear conveyor.
- Parameter:** direction: pose vector, to determine the direction of the conveyor in the base frame of the robot, list type data;
ticks_per_meter: the number of ticks of the encoder change when the conveyor moves one meter, float type data;
encoder_index: The encoder serial number associated with the conveyor tracking, and the ordinal number value must be 0 or 1. When no ordinal value is specified, use the default ordinal value of 0, integer type data (optional parameter).
- Return:** None
- Example:**
- ```
track_conveyor_linear([1,0,0,0,0,0],1000.0)
Parameters:
direction=[1,0,0,0,0,0]
ticks_per_meter=1000.0
encoder_index, =0The default value
```

### 3.1.32 stop\_conveyor\_tracking command

```
stop_conveyor_tracking()
```

- Function:** This command stops the conveyor tracking function triggered by track\_conveyor\_linear() or track\_conveyor\_circular(), and decelerate the tool speed to zero.
- Parameter:** None
- Return:** None
- Example:**
- ```
stop_conveyor_tracking()
```

3.1.33 get_actual_tool_flange_pose command

```
get_actual_tool_flange_pose()
```

Function: This command is used to obtain the current tool flange pose.

Parameter: None

Return: The current measured tool flange pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example: `get_actual_tool_flange_pose()`
Return value: The current tool flange pose

3.1.34 get_target_waypoint command

```
get_target_waypoint()
```

Function: This command is used to obtain the target waypoint is currently moving to.

Parameter: None

Return: The current target waypoint, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example: `get_target_waypoint()`
Return value: The target waypoint of the active move

3.1.35 is_within_safety_limits

```
is_within_safety_limits(p, qNear='', ciPair='') Or is_within_safety_limits(qNear='', qNear='', ciPair='')
```

Function: This command is used to verify if the target pose or joint angle is reachable by detecting if the given parameters are within the currently active robot safety limits. A boolean result will be returned. This command can be used for pre-checks before the robot moves.

Two ways can be used to refer to the target point, pose or joint angle. Select one of them as the position parameter of the target point.

Four aspects will be detected: robot joint limits, robot safety plane limits, robot safety tool limits, and robot forward/inverse kinematics limits.

Parameter: p: TCP pose of the robot's end, whose format is [x, y, z, Rx, Ry, Rz], list type data (unit: x/y/z is m, Rx/Ry/Rz is rad); Use the list without any keyword to refer to pose.

q: joint angle, whose format is [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], list type data (unit: rad).

The parameter with "q=" keyword means joint angle. qnear: the closest joint angle (optional parameter), whose format is [Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3], list type data (unit: rad); The default value is the robot's real-time joint angle, list type data. This parameter will be used for inverse kinematics calculation only when the first parameter represents pose.

ciPair: the status of the configurable i/o pairs of the safe plane (optional parameter), whose format is [1,1,1,1], list type data, the value of each digit is either 0 or 1.

The first digit is 1 = to check for security planes configured in CI0 && CI1 and with a valid security plane; 0 = No check;

The second digit is 1 = to check for security planes configured in CI2 && CI3 and with a valid security plane; 0 = No check;

The third digit is 1 = to check for security planes configured in CI4 && CI5 and with a valid security plane; 0 = No check;

The first digit is 1 = to check for security planes configured in CI6 && CI7 and with a valid security plane; 0 = No check;

If this parameter has no value, the system will detect the safe plane based on the status of the actual CI pairs and the validity of safe planes by default.

When the safe plane is valid and there is no any CI pair configured, the safe plane is detected by default.

Return: True = within the safety limit and the robot can safely move to the target position; False = out of the safety limit.

Example: `is_within_safety_limits(p, qNear='', ciPair='')`
Return value: True

NOTICE

Since p and q parameters share the same data format, they need to distinguished by the following:

1. The data with "q=" keyword refers to joint angle, for instance, q=[Base, Shoulder, Elbow, Wrist1, Wrist2, Wrist3];
2. The data without any keyword represents pose parameter, for instance, [x, y, z, Rx, Ry, Rz].

3.1.36 get_tcp_offset command

```
get_tcp_offset()
```

Function: This command is used to obtain the current TCP offset.

Parameter: None

Return: The current TCP offset in the format [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
get_tcp_offset()
```


Return value: The current target tool offset

3.1.37 load_micro_line_file command

```
load_micro_line_file(file_name, interval_time, pos_type=0, unit_type=0)
```

- Function:** This command is used to load the micro-segment file through the micro-segment file name, set the time interval between two adjacent points of the micro-segment, the data type of the micro-segment file, and the parameters of the unit type of the micro-segment file, then generate and return a unique identification number.
- Parameter:** `file_name`: micro-segment file name, string type data;
`interval_time`: the time interval between two points in a micro-segment file, in ms, integer type data;
`pos_type`: micro-segment file data type, integer type data: 0: joint space data, 1: Cartesian data (optional parameters);
`unit_type`: micro-segment file unit type, integer type data: 0: joint space data is radians, Cartesian data is meters and radians, 1: joint space data is degrees, Cartesian data is millimeters and radians (optional parameters).
- Return:** Integer type data, the current micro-segment file and the mini-segment sequence number loaded under the corresponding configuration.
- Example:**
- ```
load_micro_line_file("new_pose1",2)
Parameters:
new_pose1="new_pose1"
interval_time=2
Return value: Micro-segment serial number
```

### 3.1.38 `micro_line_set_pcs` command

```
micro_line_set_pcs(micro_line_index, part_coordinate_system)
```

- Function:** This command is used to set the local coordinates of the current micro-segment sequence number corresponding to the micro-segment file;  
See also: 2.1.34 `load_micro_line_file()` command, `micro_line_index` are usually obtained by `load_micro_line_file()`.
- Parameter:** `micro_line_index`: micro-segment ordinal number, integer type data;  
`part_coordinate_system`: local coordinates, list type data.
- Return:** None
- Example:**
- ```
micro_line_set_pcs(0,[1,0,0,0,0,0])
```

3.1.39 `get_micro_line_values_by_index` command

```
get_micro_line_values_by_index(micro_line_index, pos_type, line_index)
```

Function: This command returns the data corresponding to the micro-segment file in the form of the joint space data or Cartesian data; See also: 2.1.34 `load_micro_line_file()` command, `micro_line_index` are usually obtained by `load_micro_line_file()`.

Parameter: `micro_line_index`: micro-segment ordinal number, integer type data;
`pos_type`: data type obtained, integer type data: 0: joint space data, 1: Cartesian data (optional parameters);
`line_index`: micro-segment file line sequence number, integer type data.

Return: List type data, joint space data or Cartesian data.

Example: `get_micro_line_values_by_index(0,0,1)`
Return value: The ordinal number 1 corresponds to the joint angle of the first line of the micro-segment file

3.1.40 moveml command

```
moveml(micro_line_index, start_line=0, end_line=0)
```

Function: This command is used to run the micro-segment file corresponding to the micro-segment sequence number, running from the set start line to the set end line;
See also: 2.1.34 `load_micro_line_file()` command, `micro_line_index` are usually obtained by `load_micro_line_file()`.

Parameter: `micro_line_index`: micro-segment ordinal number, integer type data;
`start_line`: the starting line of the run, the default is 0 for running from the first row of the micro-segment file, integer type data (optional parameter);
`end_line`: the end line of the run, the default is 0 to run to the last line of the micro-segment file, integer type data (optional parameter).

Return: None

Example: `moveml(0)`

3.1.41 freedrive_mode command

```
freedrive_mode(freeAxes=[1,1,1,1,1,1], frame=[0,0,0,0,0,0])
```

Function: This command is used to enable the freedrive mode, which allows the robot to be hand guided freely. With default parameters, the robot will be in fully free hand-guiding mode. And the robot can be in the hand-guiding mode with locking axes by modifying parameters.

Parameter: freeAxes: the data of length 6, each one is 0 or 1, 0: can not be hand guided, 1: can be hand guided. The first three data indicate the enabling state of hand-guiding along the x, y, and z direction of Cartesian space; the last three indicate the enabling state of hand-guiding rotating around the rx, ry, and rz axes of Cartesian space. This parameter indicates the state in the selected coordinate system space and it is optional.

frame: RPY position data, defines the coordinate system data (relative to the robot coordinate system) for the hand-guiding mode. This parameter can also be used with base coordinate system and tool coordinate system to indicate the robot coordinate system and the dynamic TCP coordinate system. This parameter is optional.

Return: None

Example:

```
freedrive_mode(freeAxes=[0,1,0,0,0,0], frame="tool")
Parameters:
freeAxes = [0,1,0,0,0,0] -> Hand guide the robot along the Y axis
frame = "tool" -> Based on the real time TCP coordinate system
which is activated
```

3.1.42 end_freedrive_mode command

```
end_freedrive_mode()
```

Function: This command is used to disable the hand-guiding mode and makes the robot be in the position mode. If the running task is finished, the robot will exit hand-guiding mode automatically.

Parameter: None

Return: None

Example:

```
end_freedrive_mode()
Disable hand-guiding mode
```

3.1.43 set_collision_detect_enabled command


```
set_collision_detect_enabled()
```

Function: This command is used to enable/disable the collision detection.

Parameter: Boolean. True = enabled; False = disabled.

Return: None

Example:

```
set_collision_detect_enabled(True)
This example enables collision detection.
set_collision_detect_enabled(False)
This example disables collision detection.
```

3.1.44 is_collision_detect_enabled command

```
is_collision_detect_enabled()
```

Function: This command returns whether collision detection is currently enabled.

Parameter: None

Return: Boolean. True = enabled; False = disabled.

Example:

```
is_collision_detect_enabled()
Return Value: True or False
```

3.1.45 get_dyn_collision_force_min_ratio command

```
get_dyn_collision_force_min_ratio()
```

Function: This command is used to obtain the current collision detection sensitivity value.

Parameter: None

Return: Integer value (percentage).

Example:

```
get_dyn_collision_force_min_ratio()
Return Value: 50 (indicating that the current collision detection
sensitivity is 50%).
```

3.1.46 set_dyn_collision_force_min_ratio command

```
set_dyn_collision_force_min_ratio(int ratio)
```

Function: This command is used to set the collision detection sensitivity.

Parameter: int ratio: Collision detection sensitivity (percentage), range: [10, 100].

Return: None

Example: `set_dyn_collision_force_min_ratio(20)`
This example sets the collision detection sensitivity to 20%.

3.2 MATH-related command

This section mainly introduces computation-related command.

3.2.1 get_inverse_kin command

```
get_inverse_kin(p, qnear='', tcp='active tcp')
```

Function: This command is used for solving the inverse kinematic functions, i.e. convert Cartesian coordinates into the joint space coordinates. If qnear is defined, the solution closest to qnear is returned. Otherwise, the solution closest to the current joint positions is returned. If no tcp is provided the currently active tcp of the controller will be used.

Parameter: p: pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
qnear: the closest joint angle, the default value is the current robot joint angle, list type data (optional parameter);
tcp: tcp offset pose (optional parameter).

Return: List type data, joint angle, unit: rad.

Example: `global v`
`po=[0.4,-0.2,0.4,-3.1,2.8,-1.8]`
`qn=[-0.2,-1.6,-1.4,-1.5,1.5,-1.6]`
`v=get_inverse_kin(po,qnear=qn)`
Parameters:
`p=po=[0.4,-0.2,0.4,-3.1,2.8,-1.8]`
`qnear=qn=[-0.2,-1.6,-1.4,-1.5,1.5,-1.6]`

3.2.2 get_inverse_kin_has_solution command

```
get_inverse_kin_has_solution(p, qnear='', tcp='active tcp')
```

Function: This command is used for checking if the function `get_inverse_kin` has a solution and returning to a boolean type result (True or False). It can avoid triggering the running alarm when there is no solution for the function `get_inverse_kin`.

Parameter: `p`: pose, the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` units are m, `Rx, Ry, Rz` units are rad;
`qnear`: the closest joint angle, the default value is the current robot joint angle, list type data (optional parameter);
`tcp`: tcp offset pose, the default value is the current activation tool of the system (optional parameter).

Return: Whether there is a solution or not, boolean type data, True or False.

Example:

```
global v
p=[0.41951,-0.16,0.25745,-3.11757,0,-1.5708]
v=get_inverse_kin_has_solution(p)
print(v)
Parameters:
p=po=[0.41951,-0.16,0.25745,-3.11757,0,-1.5708]
Return value: True
```

3.2.3 get_all_inverse_kin command

```
get_all_inverse_kin(p, tcp='active_tcp')
```

Function: This command is used to get all inverse solution data.

Parameter: `p`: pose, the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` units are m, `Rx, Ry, Rz` units are rad;

`tcp`: tcp offset pose (optional parameter).

Return: List type data, list member is a list object of joint angle, unit of joint angle: rad, range $[-\pi, \pi]$, length of return value indicates the number of inverse solution results.

Example:

```
global v
v = get_all_inverse_kin([0.4771486056398243,0.009603682884449335,
0.5119841047483048,-3.111929473888252,4.519127222985093E-6,
-1.570799425431062], tcp=[0,0,0,0,0,0])
global ret_count
ret_count = len(v)

Return value:
v: [0.336318, -3.013889, 1.620656, 2.992033, 1.561013, 0.336185],
    [0.336318, -1.487028, -1.620656, -1.576702, 1.561013, 0.336185],
    [0.336318, -2.965255, 1.062955, 0.359507, -1.561013, -2.805408],
    [0.336318, -1.954786, -1.062955, 1.474948, -1.561013, -2.805408],
    [2.845753, -1.186798, 1.06285, 1.666371, 1.562145, 2.845879],
    [2.845753, -0.176428, -1.06285, 2.7817, 1.562145, 2.845879],
    [2.845753, -1.654565, 1.620748, -1.565353, -1.562145, -0.295714],
    [2.845753, -0.127621, -1.620748, 0.149199, -1.562145, -0.295714]
ret_count : 8 It indicates that there are 8 inverse solution results.
```

Note: If the inverse solution fails, v is an empty list and ret_count is 0.

3.2.4 get_forward_kin command

```
get_forward_kin(q='', tcp='active tcp')
```

Function: This command is used for solving the forward kinematics functions, i.e. convert joint space coordinates into the Cartesian coordinates. If no joint position vector is provided the current joint angles of the robot arm will be used. If no tcp is provided the currently active tcp of the controller will be used.

Parameter: q: robot joint angle, unit is rad, list type data (optional parameter);
tcp: tcp offset pose(optional parameter).

Return: Robot pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
global p_p
p_j=[-0.2,-1.6,-1.4,-1.5,1.5,0]
p_p=get_forward_kin(p_j)
Parameters:
q=p_j=[-0.2,-1.6,-1.4,-1.5,1.5,0]
```

3.2.5 bit_list_to_integer command

```
bit_list_to_integer(l)
```

Function: This command is used to calculate and return the integer value represented by the boolean value contained in the list l.

Parameter: l: The list of booleans to be converted to an integer. boolean value at index 0 as least significant bit. False represents 0 and True represents 1. If the list is empty, this function returns 0. If the list contains more than 32 boolean values, this function returns the signed integer values converted by the first 32 boolean values in the list, list type data.

Return: Integer type data, an integer value represented by a binary list.

Example: `bit_list_to_integer([True , 4<5, 9==10 , 9%3==0])`
Parameter: l = [True , 4<5, 9==10 , 9%3==0] -> this list of data is used to be converted
Return value: 11

3.2.6 integer_to_bit_list command

```
integer_to_bit_list(x)
```

Function: This command is used to calculate and return a list of boolean values as a binary number of signed integer values x.

Parameter: x: To convert an integer value to a binary list, the return value contains a list of 32 boolean values, where False represents 0, True represents 1, list type data.

Return: List type data, binary boolean list, the value at list index 0 is the lowest bit of the input value x.

Example: `integer_to_bit_list(11)`
Parameter: x = 11 -> this integer is used to be converted.
Return value: [True, True, False, True, ..., False, False](Length is 32)

3.2.7 get_list_length command

```
get_list_length(v)
```

Function: This command is used to calculate and return the length of a list variable, which is the number of entries the list contains.

Parameter: v: list variable, list type data.

Return: Integer type data, an integer that specifies the length of a given list.

Example:

```
list=[666,666,666,666,666,666,666,666,666,666]
list_length=get_list_length(list)
Return value:10
```

3.2.8 length command

length(v)

Function: This command is used to calculate and return the length of a list variable or string variable.

Parameter: v: A list or string variable, string type data or list type data.

Return: Integer type data, list or string length.

Example:

```
length([1,2,3,4,5])
Return value:5
length("hello world")
Return value:11
```

3.2.9 acos command

acos(f)

Function: This command is used to calculate and return the arccosine value of the parameter f, and the return value is in rad.

Parameter: f: The cosine value of the angle to be calculated, the defined domain is: [-1 , 1], float type data.

Return: Float type data, the arccosine value of f, in rad.

Example:

```
acos(0.866)
Return value:0.5235 rad
```

3.2.10 asin command

asin(f)

Function: This command is used to calculate and return the arcsine value of the parameter f , the return value unit is rad.

Parameter: f : The sine value of the angle to be calculated, the defined domain is: $[-1, 1]$, float type data.

Return: Float type data, the arcsine value of f , in rad.

Example:

```
asin(0.5)
Return value:0.5235 rad
```

3.2.11 atan command

```
atan(f)
```

Function: This command is used to calculate and return the arctangent value of the parameter f , and the return value is in rad.

Parameter: f : The tangent value of the angle to be calculated, float type data.

Return: Float type data, the arctangent value of f , in rad.

Example:

```
atan(1.0)
Return value:0.785 rad
```

3.2.12 atan2 command

```
atan2(x, y)
```

Function: This command is used to calculate and return the arctangent of x/y , the return value units are rad, and the signs of the x and y values determine the correct quadrant.

Parameter: x : the value used for calculation, float type data;
 y : the value used for calculation, float type data.

Return: Float type data, the arctangent of x/y , in rad.

Example:

```
atan2(1.0,1.0)
Parameters:
x=1.0, the projection of the vector in the positive direction of
the x-axis
y=1.0, the projection of the vector in the positive direction of
the y-axis
Return value:0.785 rad
```

3.2.13 cos command

cos(f)

Function: This command is used to calculate and return the cosine of the parameter f radian angle.

Parameter: f: The value to be calculated, float type data.

Return: Float type data, cosine value of f.

Example:
cos(3.1415926)
Return value:-1

3.2.14 sin command

sin(f)

Function: This command is used to calculate and return the sine of the parameter f radian angle.

Parameter: f: the value to be calculated, float type data.

Return: Float type data, sinusoidal value of f.

Example:
sin(3.1415926)
Return value:0

3.2.15 tan command

tan(f)

Function: This command is used to calculate and return the tangent of the radian angle of the parameter f.

Parameter: f: the value to be calculated, float type data.

Return: Float type data, tangent of f.

Example:
tan(0)
Return value:0

3.2.16 d2r command

d2r(d)

Function: This command is used to convert the unit of angular value d from degrees to radians.

Parameter: d : The angle in degrees, float type data.

Return: Float type data, the angle in rad.

Example: `d2r(90)`
Return value: 1.5707963267949

3.2.17 r2d command

`r2d(r)`

Function: This command is used to convert the unit of angular value r from radians to degrees.

Parameter: r : radians in rad, float type data.

Return: Float type data, the angle in degrees.

Example: `r2d(3.1415926)`
Return value: 180

3.2.18 ceil command

`ceil(f)`

Function: This command returns the smallest integer not less than f . The integer rounds up.

Parameter: f : The value to be calculated, float type data.

Return: Integer type data, rounded integer.

Example: `ceil(-8.8)`
Return value: -8

3.2.19 floor command

`floor(f)`

Function: This command returns the largest integer not greater than f. The integer rounds down.

Parameter: f: The input floating-point value, float type data.

Return: Integer type data, integer rounds down from f.

Example: `floor(8.8)`
Return value:8

3.2.20 log command

```
log(b, f)
```

Function: This command calculate base b logarithm of f. If b or f is negative, an error will be reported during running.

Parameter: b: base number, float type data;
f: true number, float type data.

Return: Float type data, with b as the logarithm of f.

Example: `log(2,3)`
Return value:1.584

3.2.21 sqrt command

```
sqrt(f)
```

Function: This command calculate square root of f. If b or f is negative, an error will be reported during running.

Parameter: f: the value to be calculated, float type data.

Return: Float type data, the square root of f.

Example: `sqrt(4)`
Return value:2

3.2.22 pow command

```
pow(base, exponent)
```

Function: This command calculates the exponentiation with the given base and exponent values.

Parameter: base: base value, float type data;
exponent: exponential value, float type data.

Return: Float type data, the result of the power operation.

Example: `pow(2,3)`
Return value:8

3.2.23 norm command

```
norm(a)
```

Function: This command is used to calculate and return the norm of variables;
The variables can be one of four different types:
pose: in this case, returns the Euclidean norm of the pose;
Floating-point number: in this case, returns the absolute value (a);
Integer: In this case, the absolute value (a) is returned;
List: In this case, returns the Euclidean norm of the list. The list elements must be numeric.

Parameter: a: Pose, floating-point number, integer or list, list type data, integer type data or float type data.

Return: Float type data, the norm of a.

Example: `norm(-10)`
Return value:10

3.2.24 normalize command

```
normalize(v)
```

Function: This command normalizes the data in a given list of floating-point numbers. If all elements in the list are 0, an error will be reported during running.

Parameter: v: A given list of floating-point numbers, list type data.

Return: List type data, an array of all data united.

Example:

```
As normalize([0,1,0])
Return value:[0,1,0]
As normalize([2,0,0])
Return value:[1,0,0]
As normalize([1,1])
Return value:[0.707,0.707]
```

3.2.25 point_dist command

```
point_dist(p_from, p_to)
```

Function: This command is used to calculate the distance from the point p_from to the point p_to.

Parameter: p_from: the starting pose of the tool, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
p_to: the target pose of the tool, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: Float type data, distance between two tool positions (rotation is not taken into account).

Example:

```
global p_dist
pf=[0.4,-0.2,0.4,-3.1,2.8,-2.3]
pt=[-0.01,-1.5,-0.002,-1.5,1.5,-2.3]
p_dist=point_dist(pf, pt)
Return value:1.0
```

3.2.26 pose_add command

```
pose_add(p_1, p_2)
```

Function: This command is used for summing point p_1 and point p_2 position and pose. P_1 and p_2 contain three positional parameters (x, y, z), collectively referred to as P, and three rotational parameters (Rx, Ry, Rz), collectively referred to as R. This function is used to calculate the result of a given pose addition p_3, as follows:

$$p_3.P = p_1.P + p_2.P$$

$$p_3.R = p_1.R * p_2.R$$

Parameter: p_1: tool pose 1, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
p_2: tool pose 2, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: A list of the sum of the products of the position part and the rotated part, in the form of [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
global add_pose
p1=[0.4,-0.2,0.4,-3.1,2.8,-2.3]
p2=[-0.01,-1.5,-0.002,-1.5,1.5,-2.3]
add_pose=pose_add(p1,p2)
Return value:[0.39,-1.7,0.398,-2.3149,1.1819,-2.27039]
```

3.2.27 pose_dist command

```
pose_dist(p_from, p_to)
```

Function: This command is used to calculate and return the pose distance between the point p_from and the point p_to.

Parameter: p_from: The starting pose of the tool, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
p_to: tool target pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: Float type data, the distance between two pose, including rotation.

Example:

```
global p_dist
pf=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
pt=[-0.01, -1.5, -0.002, -1.5, 1.5, -2.3]
p_dist=pose_dist(pf, pt)
Parameters:
p_from=pf=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
p_to=pt=[-0.01, -1.5, -0.002, -1.5, 1.5, -2.3]
Return value:2.50394
```

3.2.28 pose_inv command

```
pose_inv(p_from)
```

Function: This command is used to calculate and return the inverse of the pose.

Parameter: p_from: tool pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: The inverse of p_from is in the form of [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
global inv_pose
pf=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
inv_pose=pose_inv(pf)
Return value:[0.023407, 0.41385, -0.43378, 0.23146, -0.25682, -
0.86443]
```

3.2.29 pose_sub command

```
pose_sub(p_to, p_from)
```

Function: This command is used for the subtraction of two-point poses of point p_to and point p_from. P_to and p_from contain three positional parameters (x, y, z), collectively referred to as P, and three rotational parameters (Rx, Ry, Rz), collectively referred to as R. This function is used to calculate the result of a given pose subtraction p_result, as follows:

$$p_result.P = p_to.P - p_from.P$$

$$p_result.R = (p_from.R)^{-1} * p_to.R$$

Parameter: p_to: tool pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;

p_from: tool pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Return: Tool pose transformation, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.

Example:

```
global sub_pose
pt=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
pf=[-0.01, -1.5, -0.002, -1.5, 1.5, -2.3]
sub_pose=pose_sub(pt, pf)
Return value:[0.41, 1.3, 0.402, -1.54817, 0.06821, 1.29935]
```

3.2.30 pose_trans command

```
pose_trans(p_from, p_from_to)
```

Function: The first argument `p_from` used to transform the second argument the `p_from_to`. The result will be returned. The result of the pose is calculated by moving the coordinate system of `p_from` to position of `p_from_to`, then rotating the coordinate system by `Rx`, `Ry`, `Rz` of `p_from_to`. If the pose is treated as a transformation matrix, it looks like this:

$$T_{\text{world} \rightarrow \text{to}} = T_{\text{world} \rightarrow \text{from}} * T_{\text{from} \rightarrow \text{to}}$$
$$T_{x \rightarrow \text{to}} = T_{x \rightarrow \text{from}} * T_{\text{from} \rightarrow \text{to}}$$

Parameter: `p_from`: starting pose, the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` units are `m`, `Rx, Ry, Rz` units are `rad`;
`p_from_to`: the pose change relative to the starting pose, the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` units are `m`, `Rx, Ry, Rz` units are `rad`.

Return: The generated pose, the format is `[x, y, z, Rx, Ry, Rz]`, list type data, where `x, y, z` units are `m`, `Rx, Ry, Rz` units are `rad`.

Example:

```
global tp
pf=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
pft=[-0.01, -1.5, -0.002, -1.5, 1.5, -2.3]
tp=pose_trans(pf, pft)
Return value:[1.4, -1.2, 0.3, -2.3, 1.1, -2.2]
```

3.2.31 rotvec2rpy command

```
rotvec2rpy(rotation_vector)
```

- Function:** This command is used to calculate the RPY vector corresponding to the rotation_vector.
- Parameter:** rotation_vector: rotation vector (Vector3d) radian, also called the Axis-angle vector (rotation angle multiplied by radians per unit of rotation axis), list type data.
- Return:** List type data, an RPY vector in radians (Vector3d), describing a roll-pitch-yaw sequence axis of external rotation around X-Y-Z, (as opposed to the inherent axis of rotation around Z-Y'-X''). In matrix form, the RPY vector is defined as $R_{rpy} = R_z(\text{yaw})R_y(\text{pitch})R_x(\text{roll})$.
- Example:**
- ```
rotvec2rpy([3.14, 1.57, 0])
Return value: [-2.8, -0.1, 0.9]
```

### 3.2.32 rpy2rotvec command

```
rpy2rotvec(rpy_vector)
```

- Function:** This command is used to calculate and return rotation vectors corresponding to the parameter rpy\_vector.
- Parameter:** rpy\_vector: An RPY vector in radians (Vector3d), describing a roll-pitch-yaw sequence axis of external rotation around X-Y-Z, (relative to the inherent axis of rotation around Z-Y'-X''). In matrix form, the RPY vector is defined as  $R_{rpy} = R_z(\text{yaw})R_y(\text{pitch})R_x(\text{roll})$ , list type data.
- Return:** List type data, a rotation vector in radians (Vector3d), also known as axis-Angle vectors (rotation angles multiplied by radians per axis of rotation).
- Example:**
- ```
rpy2rotvec([3.14, 1.57, 0])
Return value: [2.2, 0.001, -2.2]
```

3.2.33 interpolate_pose command

```
interpolate_pose(p_from, p_to, alpha)
```


- Function:** This command is used for linear interpolation of tool positions and orientations;
When alpha is 0, p_from is returned. When alpha is 1, p_to returned;
When alpha changes from 0 to 1, it returns the straight-line pose from p_from to p_to (and changes in geodetic orientation);
If alpha is less than 0, returns the point before the p_from on the line;
If alpha is greater than 1, return to the position after p_to on the line.
- Parameter:** p_from: the starting pose of the tool, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
p_to: the target pose of the tool, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad;
alpha: floating-point number, float type data.
- Return:** Interpolated pose, the format is [x, y, z, Rx, Ry, Rz], list type data, where x, y, z units are m, Rx, Ry, Rz units are rad.
- Example:**
- ```
interpolate_pose(pf, pt, al)
pf=[0.4, -0.2, 0.4, -3.1, 2.8, -2.3]
pt=[-0.01,-1.5,-0.002, -1.5,1.5, -2.3]
al=0
Return value:[0.4, -0.2, 0.4, 0.041, 0.3, 0.8]
```

### 3.2.34 random command

```
random()
```

- Function:** This command is used to generate a random number with a value range of [0, 1].
- Parameter:** None
- Return:** Float type data, pseudo-random number between 0 and 1.
- Example:**
- ```
random()
Return value: A random number between 0 and 1
```


Chapter 4 Force Command

4.1 force_mode command

```
force_mode(reference_frame, selection_vector, target_wrench, mode, speed_limits)
```

Function: This command is used to enable force control mode and the robot will be controlled in the force control mode.

Parameter:

reference_frame: a pose vector that defines the force reference frame relative to the base frame. The format is [X,Y,Z,Rx,Ry,Rz], where X, Y, and Z represent position with the unit of m, Rx, Ry, and Rz represent pose with the unit of rad which is defined by standard Euler angles.

selection_vector: a 6-dimensional vector consisting of 0 and 1 that defines the compliant axis in the force frame. 1 represents the axis is compliant and 0 represents the axis is noncompliant.

target_wrench: the force/torque applied to the environment by the robot. The robot moves/rotates along the compliant axis to adjust its pose to achieve the target force/torque. The format is [Fx,Fy,Fz,Mx,My,Mz], where Fx, Fy, and Fz represent the force applied along the compliant axis with the unit of N, Mx, My, and Mz represent the torque applied about the compliant axis with the unit of Nm. This value is invalid for the non-compliant axis. Due to the safety restrictions of joints, the actual applied force/torque is lower than the set one. In the separate thread, the command `get_tcp_force` may be used to read the actual force/torque applied to the environment.

mode: the parameter for force control mode, integer type data, ranges from 0 to 3. This parameter is used to define force control mode, i.e., how to define the force frame or how the force frame is transformed from the force reference frame.

0: Fixed mode. The force frame is the force reference frame.

1: Point mode. The Y-axis in the force frame points from the TCP origin of the robot to the origin of the force reference frame.

2: Motion mode. The X-axis in the force frame is the projection of the TCP motion direction vector in the X-Y plane belonging to the force reference frame.

3: TCP mode. The force frame is the TCP frame.

speed_limits: the parameter for the speed limit, 6-dimensional vector, float type data. The format is $[V_x, V_y, V_z, \omega_x, \omega_y, \omega_z]$, where V_x , V_y , and V_z represent the maximum speed for TCP along the compliant axis with the unit of m/s, ω_x , ω_y , and ω_z represent the maximum speed for TCP about the compliant axis with the unit of rad/s. This parameter is invalid for the non-compliant axis whose trajectory will be as set before.

Return: None

Example: `force_mode([0.5,0.8,0,0,0,1.57], [0,0,1,0,0,0], [0,0,50,0,0,0], 0, [0,0,0.1,0,0,0])`
Parameters:
Reference frame=[0.5,0.8,0,0,0,1.57] Force frame is offset from the base frame 500 mm in the direction of X-axis, 800 mm in the direction of Y-axis, and rotated 90° in the direction of RZ.
Selection Vector=[0,0,1,0,0,0] The Z-axis in the force frame is compliant and other axes are non-compliant.
Target wrench=[0,0,50,0,0,0] The robot will apply 50 N in the direction of Z-axis in the force frame.
Mode=0 The force control mode is fixed mode and the force frame is the force reference frame that is set by the user.
Speed_limits = [0,0,0.1,0,0,0] The maximum speed is 100 mm/s in the direction of the compliant Z-axis.

Note:

1. Do not perform movements with high deceleration or parallel to compliant axes before the force control mode is enabled. It is recommended to add the command sleep (at least 2 s) before the force control mode is enabled to tackle this issue.
2. Do not perform movements with high acceleration during force control mode enabled. Otherwise, they will decrease force control accuracy.

4.2 force_mode_set_gain_scaling command

```
force_mode_set_gain_scaling(scaling)
```

Function: This command is used to set the gain scaling parameter for the force control mode.

Parameter:

scaling: the gain scaling parameter, ranges from 0 to 2. The default value is 0.2.

The smaller the gain scaling value is, the slower the force tracking responses are, and the smaller the maximum deviation value is; conversely, the greater the gain scaling value is, the faster the force tracking responses are, and the greater the maximum deviation value is. But the robot will jitter with too-great gain scaling value. For example, if the gain scaling value is greater than 1, the force control mode will be unstable in case collisions happen or the robot squeezes hard surfaces.

Return: None

Example: `force_mode_set_gain_scaling(0.5)`
Set the gain scaling parameter as 0.5 for force control mode.

Note: The gain scaling value is retained until this command is called again. Please add this command at the beginning of your program or before entering force mode to ensure it is executed prior to activating force mode; otherwise, after a power-off and restart, this gain scaling value will be restored to the default.

4.3 force_mode_set_damping command

```
force_mode_set_damping(damping)
```

Function: This command is used to set the damping parameter for the force control mode.

Parameter: damping: the damping parameter, ranges from 0 to 1. The default value is 0.

When this parameter is set as 1, it is full damping. The robot will decelerate quickly if there is no external force. When this parameter is set as 0, there is no damping. The robot will move at the current speed.

The smaller the damping value is, the weaker damping in the direction of compliant axes is, the weaker the force tracking oscillation restraints are, and the faster the force tracking responses are; conversely, the greater the damping value is, the stronger damping in the direction of compliant axes is, the stronger the force tracking oscillation restraints are, and the slower the force tracking responses are.

Return: None

Example: `force_mode_set_damping(0.5)`
Set the damping parameter as 0.5 for the force control mode.

Note: The damping coefficient is retained until this command is called again. Please add this command at the beginning of your program or before entering force mode to ensure it is executed prior to activating force mode; otherwise, after a power-off and restart, the damping coefficient will be restored to the default.

4.4 zero_ftsensor command

```
zero_ftsensor()
```

Function: This command is used to zero the force/torque applied to the TCP measured by the sensor (tare sensor), whose value is the vector of the force/torque applied to the TCP and acquired from the command `get_tcp_force(True)`. This vector has been compensated. The current measured force/torque value will be saved as the referenced one after this command is performed. The referenced one will be subtracted from the subsequent measured force/torque value (tare sensor).
Notice: The referenced force/torque value mentioned above will be renewed when this command is performed. It will be reset as 0 after the controller is rebooted.

Parameter: None

Return: None

Example:

```
zero_ftsensor()  
Zero the vector of force/torque applied to the TCP which has been  
compensated (tare sensor)  
and the referenced force/torque value will be subtracted from  
subsequent measured  
force/torque value.
```

4.5 force command

`force()`

Function: This command is used to get the external force applied in the TCP. Get the external force applied in the TCP, whose value is the norm of F_x , F_y , and F_z calculated by the command `get_tcp_force(True)`. For zero the value measured by the force/torque sensor, see also 3.4 `zero_ftsensor` command.

Parameter: None

Return: The force with the unit of N, float type data

Example:

```
fe=force()  
Get the force applied in the TCP.
```

4.6 get_tcp_force command

```
get_tcp_force(enable_compensation)
```

- Function:** This command is used to get the force/torque vector applied in the TCP. For zero the value measured by the force/torque sensor, see also 3.4 zero_ftsensor command.
- Parameter:** enable_compensation: boolean type data. True: the return value will be compensated; False: the return value is the raw data measured by the force/torque sensor.
- Return:** Force/torque vector, whose format is [Fx(N),Fy(N),Fz(N),TRx(Nm),TRy(Nm),TRz(Nm)], where Fx, Fy, and Fz represent force with the unit of N, TRx, Try, and TRz represent torque with the unit of Nm. The TCP frame is the reference frame of this force/torque vector.
- Example:**
- ```
ft_comp=get_tcp_force(True)
Get the force/torque vector applied in the TCP, whose value is compensated.
ft_orig=get_tcp_force(False)
Get the force/torque measured by the sensor, whose value is raw.
```

## 4.7 end\_force\_mode command

```
end_force_mode()
```

- Function:** This command is used to disable the force control mode. It also will be performed when the procedure ends.
- Parameter:** None
- Return:** None
- Example:**
- ```
end_force_mode()
Disable the force control mode
```

4.8 Coordinate System Transformation

```
w_to=wrench_trans(T_from_to, w_from)
```


Function: This command is used to transform a wrench (force and moment) defined in the source coordinate system to the target coordinate system.

Parameter: T_from_to: the RPY pose corresponding to the coordinate system transformation matrix, in the format [X, Y, Z, Rx, Ry, Rz], where the first three components are in meters (m) and the last three ones are in radians (rad).

w_from: the force/moment (wrench) defined in the source coordinate system, in the format [Fx, Fy, Fz, Mx, My, Mz], where Fx, Fy and Fz denote forces in N, and Mx, My and Mz denote moments in Nm.

w_to: the transformed force/moment (wrench), in the format [Fx, Fy, Fz, Mx, My, Mz], where Fx, Fy, and Fz denote forces in N, and Mx, My, and Mz denote moments in Nm.

Return: The transformed force/moment vector w_to, in the format [Fx, Fy, Fz, Mx, My, Mz].

Example:

```
global w1
w1=wrench_trans([0.1, 0, 0, 0, 0, 1.5708], [10, 0, 0, 0, 0, 0])
Returned value: [0, 10, 0, 0, 0, 1]
```


Chapter 5 I/O Command

This chapter mainly introduce commands regarding I/O signal configuration.

5.1 get_configurable_digital_in command

```
get_configurable_digital_in(n)
```

Function: This command is used to obtain configurable digital input signal levels;
See also: 5.5 get_standard_digital_in command.

Parameter: n: The index of the input, integer type data: [0:7].

Return: Signal level, boolean type data, True or False.

Example: `get_configurable_digital_in(0)`
Return value: Signal level, True or False;

5.2 get_configurable_digital_out command

```
get_configurable_digital_out(n)
```

Function: This command is used to obtain configurable digital output signal levels;
See also: 5.6 get_standard_digital_out command.

Parameter: n: The index of the output, integer type data: [0:7].

Return: Signal level, boolean type data, True or False.

Example: `get_configurable_digital_out(0)`
Return value: Signal level, True or False;

5.3 get_standard_analog_in command

```
get_standard_analog_in(n)
```

Function: This command is used to obtain standard analog input signal levels;
See also: 5.7 get_tool_analog_in command.

Parameter: n: The index of the input, integer type data: [0:1].

Return: Analog signal level, float type data, in A or V.

Example: `get_standard_analog_in(1)`
Return value: analog signal level, the unit is A or V;

5.4 get_standard_analog_out command

```
get_standard_analog_out(n)
```

Function: This command is used to obtain standard analog output signal levels;
See also: 5.8 get_tool_analog_out command.

Parameter: n: The index of the output, integer type data: [0:1].

Return: Analog signal level, float type data, in A or V.

Example: `get_standard_analog_out(1)`
Return value: analog signal level, the unit is A or V;

5.5 get_standard_digital_in command

```
get_standard_digital_in(n)
```

Function: This command is used to obtain standard digital input signal levels;
See also: 5.1 get_configurable_digital_in command.

Parameter: n: The index of the input, integer type data: [0:15]. Note that MINI controller only support the range of 0-7.

Return: Signal level, boolean type data, True or False.

Example: `get_standard_digital_in(0)`
Return value: Signal level, True or False

5.6 get_standard_digital_out command

```
get_standard_digital_out(n)
```

- Function: This command is used to obtain standard digital output signal levels;
See also: 5.2 get_configurable_digital_out command.
- Parameter: n: The index of the output, integer type data: [0:15]. Note that MINI controller only support the range of 0-7.
- Return: Signal level, boolean type data, True or False.
- Example: `get_standard_digital_out(0)`
Return value: Signal level, True or False

5.7 get_tool_analog_in command

```
get_tool_analog_in()
```

- Function: This command is used to obtain tool analog input levels;
See also: 5.3 get_standard_analog_in command.
- Parameter: None
- Return: Analog input level, float type data, in A or V.
- Example: `get_tool_analog_in()`
Return value: tool analog input level, the unit is A or V

5.8 get_tool_analog_out command

```
get_tool_analog_out()
```

- Function: This command is used to obtain the tool analog output levels;
See also: 5.4 get_standard_analog_out command.
- Parameter: None
- Return: Analog output level, float type data, in A or V.
- Example: `get_tool_analog_out()`
Return value: tool analog output level, the unit is A or V

5.9 set_standard_analog_output_domain command

```
set_standard_analog_output_domain(port, domain)
```

Function: This command is used to set domain of analog outputs;
See also: 5.15 set_tool_analog_output_domain command.

Parameter: port: the index of the analog output, integer type data: [0:1];
domain: analog output domain, integer type data: 0: current mode (4-20mA), 1: voltage mode (0-10V).

Return: None

Example: `set_standard_analog_output_domain(0,0)`

5.10 set_configurable_digital_out command

```
set_configurable_digital_out(n, b)
```

Function: This command is used to set configurable digital output signal levels;
See also: 5.13 set_standard_digital_out command.

Parameter: n: The index of the output, integer type data: [0:7];
b: Signal level, boolean type data, True or False.

Return: None

Example: `set_configurable_digital_out(0,True)`

5.11 set_standard_analog_input_domain command

```
set_standard_analog_input_domain(port, domain)
```

Function: This command is used to set the domain of the standard analog input in the control box;
See also: 5.14 set_tool_analog_input_domain command.

Parameter: port: input index, integer type data: [0:1];
domain: analog input domains, integer type data: 0 represents the current mode (4-20mA), 1 represents the voltage mode (0-10V).

Return: None

Example: `set_standard_analog_input_domain(0,0)`

5.12 set_standard_analog_out command

```
set_standard_analog_out(n, f)
```

Function: This command is used to set the relative signal level value of the standard analog output I/O.

Parameter: n: The index of the output, integer type data: [0:1];
f: relative to the analog signal level (percentage of analog output range), float type data: [0:1], if the given value is greater than 1, Then set to 1, less than 0, set to 0.

Return: None

Example:

```
set_standard_analog_out(0,1.0)
Parameters:
n=0
f, =1.0Set standard analog output values to 10V or 20mA (depending
on analog I/O mode settings)
```

5.13 set_standard_digital_out command

```
set_standard_digital_out(n, b)
```

Function: This command is used to set standard digital output I/O signal levels;
See also: 5.10 set_configurable_digital_out command.

Parameter: n: The index of the output, integer type data: [0:15]. Note that MINI controller only support the range of 0-7;
b: Signal level, boolean type data, True or False.

Return: None

Example:

```
set_standard_digital_out(0,True)
```

5.14 set_tool_analog_input_domain command

```
set_tool_analog_input_domain(domain)
```

- Function:** This command is used to set the domain of analog inputs at the end of the tool;
For control box input, see also 5.11 set_standard_analog_input_domain command.
- Parameter:** domain: analog input domains, integer type data: 0 represents the current mode (4-20mA), 1 represents the voltage mode (0-10V).
- Return:** None
- Example:** `set_tool_analog_input_domain(0)`

5.15 set_tool_analog_output_domain command

```
set_tool_analog_output_domain(domain)
```

- Function:** This command is used to set the domain of the analog output at the end of the tool;
For control box output, see also 5.9 set_standard_analog_output_domain command.
- Parameter:** domain: analog output domain, integer type data: 0 represents current mode (4-20mA), 1 represents voltage mode (0-10V).
- Return:** None
- Example:** `set_tool_analog_output_domain(0)`

5.16 set_tool_digital command

```
set_tool_digital(n, b)
```


- Function:** This command is used to set the digital I/O level of the tool corresponding to a given index.
- Parameter:** n: Index of the tool digital I/O, integer type data: [0:3];
b: Signal level, boolean type data, True or False.
- Return:** None
- Example:** `set_tool_digital(1,True)`
- Note:** If the digital I/O corresponding to the current index is disabled or non-output type, the runtime alarm of I/O type error will be triggered after using this interface setting.

5.17 get_tool_digital command

```
get_tool_digital(n)
```

- Function:** This command is used to obtain the digital I/O signal level of the tool corresponding to a given index.
- Parameter:** n: Index of the tool digital I/O, integer type data: [0:3].
- Return:** Signal level, boolean type data, True or False.
- Example:** `get_tool_digital(1)`
Return value: True or False

5.18 Set Tool Analog I/O Working Mode

```
set_tool_analog_io_work_mode(domain)
```

- Function:** This command is used to set the working mode of the tool analog I/O.
- Parameter:** domain: Working mode. Integer data type.
0: USART mode
1: Analog input enabled.
2: Analog output enabled.
3: Analog input and output enabled.
- Return:** none
- Example:** `none`

NOTICE

ES series robots only support USART mode.

5.19 Configure Tool Digital I/O

```
set_tool_digital_io_config(index, mode, enable, is_out)
```

Function: This command is used to configure the tool digital I/O.

Parameter: index: pin number of the tool digital I/O. Integer data type.

mode: working mode of the digital I/O. Integer data type.

0: push/pull.

1: PNP (sourcing).

2: NPN (sinking).

enable: whether to be enabled. Boolean data type. True = enabled; False = disabled.

is_out: whether it is an output. Boolean data type. True = output, False = input.

Return: none

Example: none

NOTICE

ES series robots only support single pin mode (all digital I/Os are enabled, digital I/O [0] and [1] are outputs with PNP mode; digital I/O [2] and [3] are inputs with PNP mode).

5.20 set_runstate_configurable_digital_output_to_value command

```
set_runstate_configurable_digital_output_to_value(port, runstate)
```

Function: This command is used to set the output signal level to follow the task running state change (run or stop);

See also:

5.19 set_runstate_gp_boolean_output_to_value command

5.22 set_runstate_standard_digital_output_to_value command

5.23 set_runstate_tool_digital_output_to_value command

Parameter: port: index of output, integer type data: [0:7];

runstate: integer type data: [0:3];

0: Maintain the signal level unchanged;

1: The signal level of the task is low when it is not running, and the signal level of the task running state is unchanged;

2: The signal level of the task is high when it is not running, and the signal level of the task running state is unchanged;

3: The signal level of the task is low when it is not running, and the signal level of the task running state is high.

Return: None

Example: `set_runstate_configurable_digital_output_to_value(1,2)`

Parameters:

port=1

runstate, =2When the task is in the non-running state, the signal level of output 1 is set to high, and the signal level of the task operation state is unchanged;

5.21 set_runstate_gp_boolean_output_to_value command

```
set_runstate_gp_boolean_output_to_value(port, runstate)
```

Function: This command is used to set the output signal level to follow the task running state change (run or stop);

See also:

5.18 set_runstate_configurable_digital_output_to_value command

5.22 set_runstate_standard_digital_output_to_value command

5.23 set_runstate_tool_digital_output_to_value command

Parameter: port: index of output, integer type data: [0:127]

runstate: integer type data: [0:3]

0: Maintain the signal level unchanged;

1: The signal level of the task is low when it is not running, and the signal level of the task running state is unchanged;

2: The signal level of the task is high when it is not running, and the signal level of the task running state is unchanged;

3: The signal level of the task is low when it is not running, and the signal level of the task running state is high.

Return: None

Example: set_runstate_gp_boolean_output_to_value(1,2)

Parameters:

port=1

runstate, =2When the task is in the non-running state, the signal level of output 1 is set to high, and the signal level of the task running state is unchanged

5.22 set_runstate_standard_analog_output_to_value command

```
set_runstate_standard_analog_output_to_value(port, runstate)
```

Function: This command is used to set the output analog signal level to change (run or stop) with the operation state of the task.

Parameter: port: index of output, integer type data: [0:1];
runstate: integer type data: [0:3];
0: Maintain the analog signal level unchanged;
1: The simulating signal level of the task is at the lowest value when it is not running, and the signal level of the task running state is unchanged;
2: The simulating signal level of the task is at the highest value when it is not running, and the signal level of the task running state is unchanged;
3: The simulating signal level of the task is at the lowest value when it is not running, and the signal level of the task running state is at the highest value;

Return: None

Example: `set_runstate_standard_analog_output_to_value(1,2)`
Parameters:
port=1
runstate, =2When the task is in the non-running state, the signal level of the analog output 1 is set to the highest value, and the signal level of the task operation state is unchanged

5.23 set_runstate_tool_analog_output_to_value command

```
set_runstate_tool_analog_output_to_value(runstate)
```

Function: This command is used to set the analog output analog signal level of the tool to change (run or stop) with the operation state of the task.

Parameter: runstate: integer type data: [0:3];
0: Maintain the analog signal level unchanged;
1: The simulating signal level of the task is at the lowest value when it is not running, and the signal level of the task running state is unchanged;
2: The simulating signal level of the task is at the highest value when it is not running, and the signal level of the task running state is unchanged;
3: The simulating signal level of the task is at the lowest value when it is not running, and the signal level of the task running state is at the highest value;

Return: None

Example:

```
set_runstate_tool_analog_output_to_value(2)
parameter:
runstate=2, when the task is in the non-running state, set the
    signal level of analog output 1 to
the highest value, and the signal level of the task running state
    remains unchanged
```

5.24 set_runstate_standard_digital_output_to_value command

```
set_runstate_standard_digital_output_to_value(port, runstate)
```

Function: This command is used to set the output signal level to follow the task running state change (run or stop);

See also:

5.18 set_runstate_configurable_digital_output_to_value command

5.19 set_runstate_gp_boolean_output_to_value command

5.23 set_runstate_tool_digital_output_to_value command

Parameter: port: index of output, integer type data; range: [0:15]. Note that MINI controller only support the range of 0-7;

runstate: integer type data: [0:3];

0: Maintain the signal level unchanged;

1: The signal level of the task is low when it is not running, and the signal level of the task running state is unchanged;

2: The signal level of the task is high when it is not running, and the signal level of the task running state is unchanged;

3: The signal level of the task is low when it is not running, and the signal level of the task running state is high.

Return: None

Example: `set_runstate_standard_digital_output_to_value(1,2)`

Parameters:

port=1

runstate, =2When the task is not running, the signal level of the standard digital output 1 is

set to high, and the signal level of the task operation state is unchanged

5.25 set_runstate_tool_digital_output_to_value command

```
set_runstate_tool_digital_output_to_value(port, runstate)
```

Function: This command is used to set the output signal level to follow the task running state change (run or stop);

See also:

5.18 set_runstate_configurable_digital_output_to_value command

5.19 set_runstate_gp_boolean_output_to_value command

5.22 set_runstate_standard_digital_output_to_value command

Parameter: port: index of output, integer type data: [0:3];

runstate: integer type data: [0:3];

0: Maintain the signal level unchanged;

1: The signal level of the task is low when it is not running, and the signal level of the task running state is unchanged;

2: The signal level of the task is high when it is not running, and the signal level of the task running state is unchanged;

3: The signal level of the task is low when it is not running, and the signal level of the task running state is high.

Return: None

Example: set_runstate_tool_digital_output_to_value(1,2)

Parameters:

port=1

runstate, =2When the task is in the non-running state, the signal level of output 1 is set to high, and the signal level of the task running state is unchanged

5.26 set_input_actions_to_default command

```
set_input_actions_to_default()
```


Function: This command is used to reset the I/O trigger action of all standard digital inputs, configurable digital inputs, tool digital inputs, and universal boolean input registers to "default";

See also:

5.26 set_standard_digital_input_action command

5.25 set_configurable_digital_input_action command

5.27 set_tool_digital_input_action command

5.28 set_gp_boolean_input_action command

Parameter: None

Return: None

Example: `set_input_actions_to_default()`
Description: Set the I/O trigger for all digital inputs and universal boolean input registers as "default".

5.27 set_configurable_digital_input_action command

set_configurable_digital_input_action(port, action)

Function: This command is used to bind configurable digital inputs to trigger actions, including default trigger actions ("default") and drag mode trigger actions ("freedrive"), and to trigger corresponding actions when the bound input signal level is high;

See also:

5.24 set_input_actions_to_default command

5.26 set_standard_digital_input_action command

5.27 set_tool_digital_input_action command

5.28 set_gp_boolean_input_action command

Parameter: port: input index, integer type data: [0:7];
action: trigger action type "default" or "freedrive", string type data.

Return: None

Example: `set_configurable_digital_input_action(0,"freedrive")`
Parameters:
port=0
action="freedrive, "When the signal level of the configurable input 0 is high, drag mode is turned on"

5.28 set_standard_digital_input_action command

```
set_standard_digital_input_action(port, action)
```

Function: This command is used to bind standard digital inputs to trigger actions, including default trigger actions ("default") and drag mode trigger actions ("freedrive"), and to trigger corresponding actions when the bound input signal level is high;

See also:

5.24 set_input_actions_to_default command

5.25 set_configurable_digital_input_action command

5.27 set_tool_digital_input_action command

5.28 set_gp_boolean_input_action command

Parameter: port: input index, integer type data: [0:15]. Note that MINI controller only support the range of 0-7;

action: trigger action type "default" or "freedrive", string type data.

Return: None

Example: `set_standard_digital_input_action(0, "freedrive")`

Parameters:

port=0

action="freedrive, "When the signal level of the standard input 0 is high, drag mode is turned on"

5.29 set_tool_digital_input_action command

```
set_tool_digital_input_action(port, action)
```

Function: This command is used to bind the tool digital input to trigger actions, including default trigger actions ("default") and drag mode trigger actions ("freedrive"), and trigger corresponding actions when the bound input signal level is high;

See also:

5.24 set_input_actions_to_default command

5.25 set_configurable_digital_input_action command

5.26 set_standard_digital_input_action command

5.28 set_gp_boolean_input_action command

Parameter: port: input index, integer type data: [0:3];
action: trigger action type "default" or "freedrive", string type data.

Return: None

Example:

```
set_tool_digital_input_action(0,"freedrive")
Parameters:
port=0
action="freedrive, "When the signal level of the tool's digital
input 0 is high, drag mode is turned on;
```

5.30 set_gp_boolean_input_action command

```
set_gp_boolean_input_action(port, action)
```

Function: This command is used to bind universal boolean input registers to trigger actions, including default trigger actions ("default") and drag mode trigger actions ("freedrive"), and trigger corresponding actions when the bound input signal level is high;

See also:

5.24 set_input_actions_to_default command

5.25 set_configurable_digital_input_action command

5.26 set_standard_digital_input_action command

5.27 set_tool_digital_input_action command

Parameter: port: input index, integer type data: [0:127];
action: trigger action type "default" or "freedrive", string type data.

Return: None

Example:

```
set_gp_boolean_input_action(0,"freedrive")
Parameters:
port=0
action="freedrive, "When the signal level of the universal boolean
input register 0 is high, drag mode is turned on"
```

5.31 tool_serial_is_open command

```
tool_serial_is_open()
```

Function: This command is used to return to the enabled or disabled state of the RS-485 communication function of the robot tool terminal.

Parameter: None

Return: The open state of serial communication, boolean type data, True is open, False is closed.

Example:

```
tool_serial_is_open()
Return value:True or False
```

5.32 tool_serial_config command

```
tool_serial_config(enabled, baud_rate, parity, stop_bits, size=8, modbus_rtu=
False, usage=0)
```

Function: This command is used to enable or disable the RS-485 communication function of the robot tool terminal, and configure the serial port.

Parameter: enabled: enabled state of tool serial port communication, boolean type data;
baud_rate: Serial port baud rate, integer type data: 9600, 19200, 38400, 57600, 115200, 1000000, 2000000;
parity: parity bit, integer type data: 0(none), 1(odd), 2(even);
stop_bits: stop bits, integer type data: 1, 2;
size: The bit width of the serial port. When not specified, the default parameter 8 is used, If modbus-rtu is enabled, this parameter is automatically set to 8;
modbus_rtu: Whether to enable modbus-rtu mode, when specified as True, it is Modbusrtu mode, when specified as False, it is RS485 mode. If not specified, use the default parameter False, boolean type data;
usage: optional parameter, integer type data: 0(Script Mode), 1(Daemon Mode). When not specified, the default value 0 is used, namely, the Script Mode. When selecting the "Script Mode", the user can access to the serial port via the script interface. Please refer to the related tool interfaces, i.e. tool_serial, tool_modbus. When selecting the "Daemon Mode", the user can access to the serial port named ttyTCIO via the ELITECO SDK Daemon plugin and open the serial port for reading and writing. If the user access to the port via the script interface, it will trigger the abnormal alarm at run-time.

Return: The open state of serial communication, boolean type data, True is open, False is closed.

Example:

```
tool_serial_config(True,115200,1,2):  
parameters  
enabled=True  
baud_rate=115200  
parity=1  
stop_bits=2  
usage=0  
Return value:True or False
```

Note: When the communication mode is Modbus-rtu, the following commands such as tool_modbus_read_registers, tool_modbus_read_bits, tool_modbus_write_registers, can be used; when the communication mode is RS485, the following commands tool_serial_read and tool_serial_write can be used.

5.33 tool_serial_read command

```
tool_serial_read(read_count, is_number, time_out)
```

Function: This command is used to read the RS-485 communication data from the robot tool terminal.

Parameter: read_count: read data length, integer type data: greater than or equal to 0;

is_number: whether to return byte data in integer format, boolean type data;

time_out: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.

Return: Bytearray or list. If the is_number is True, the return value is a list of type integer. If the is_number is False, the return value is bytes array. If a timeout occurs, all data that has been read is returned.

Example:

```
tool_serial_read(3, True, 0.1)
Return value: Read integer data list, similar to [255, 1, 0]
tool_serial_read(3, False, 0.1)
Return value: Bytes of data read, similar to b"a/x90/x33"
```

5.34 tool_serial_write command

```
tool_serial_write(write_data)
```

Function: This command is used to send out the data via the robot tool terminal RS-485 communication.

Parameter: write_data: the byte data to be sent, string type data, integer type data, integer list type data or bytearray type data, if it is integer or integer list type data, one or more integer data will be written, and if the data value exceeds the range of [0-255], it will be automatically cast into the range according to the way of memory type conversion or memory truncation.

Return: Integer type data, the length of the byte data successfully sent, less than or equal to 0, indicates that the send failed.

Example:

```
tool_serial_write("hello world")
Return value: The length of the data that was successfully sent
tool_serial_write(255)
Return value: The length of the data that was successfully sent
tool_serial_write([255, 0, 10])
Return value: The length of the data that was successfully sent
```

5.35 set_tool_voltage command

```
set_tool_voltage(voltage)
```

Function: This command is used to set the voltage of the external tool connector of the robot tool flange to provide the power supply, which can be 0, 12 or 24 V.

Parameter: voltage: External tool connector provides the voltage of the power supply, unit: volts, integer type data: 0, 12 or 24.

Return: None

Example:

```
set_tool_voltage(24)
```

5.36 get_task_path command

```
get_task_path()
```

- Function: The command is used to return to the storage directory of the EliRobot tasks.
- Parameter: None
- Return: String type data, the storage directory of the EliRobot tasks.
- Example: `task_path=get_task_path()`

5.37 read_input_boolean_register command

```
read_input_boolean_register(address)
```

- Function: This command is used to read boolean values from input registers and can also be accessed via fieldbus. This register is stored in the system memory space.
- Parameter: address: Register address. Integer type data: [0:127].
- Return: Boolean type data, the value of the selected input register.
- Example: `read_input_boolean_register(88)`
Return value: True or False

5.38 read_input_float_register command

```
read_input_float_register(address)
```

- Function: This command is used to read floating-point numbers from input registers and can also be accessed via fieldbus. This register is stored in the system memory space.
- Parameter: address: Register address. Integer type data: [0:47].
- Return: Float type data, the value of the selected input register.
- Example: `read_input_float_register(2)`
Return value: A floating-point number

5.39 read_input_integer_register command

```
read_input_integer_register(address)
```


Function: This command is used to read integers from the input registers and can also be accessed via a fieldbus. This register is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:47].

Return: Integer type data, the value of the selected input register.

Example:

```
read_input_integer_register(2)
Return value:integer
```

5.40 read_output_boolean_register command

```
read_output_boolean_register(address)
```

Function: This command is used to read boolean values from the output registers and can also be accessed via a fieldbus. This register is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:127].

Return: Boolean type data, the value of the selected output register.

Example:

```
read_output_boolean_register(88)
Return value:True or False
```

5.41 read_output_float_register command

```
read_output_float_register(address)
```

Function: This command is used to read floating-point numbers from the output registers and can also be accessed via fieldbus. This register is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:47].

Return: Float type data, the value of the selected output register.

Example:

```
read_output_float_register(2)
Return value: A floating-point number
```

5.42 read_output_integer_register command

```
read_output_integer_register(address)
```

Function: This command is used to read integers from output registers and can also be accessed via fieldbus. This register is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:47].

Return: Integer type data, the value of the selected output register.

Example:

```
read_output_integer_register(2)
Return value:integer
```

5.43 write_output_boolean_register command

```
write_output_boolean_register(address, value)
```

Function: This command is used to write boolean values to an output register which is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:127];
value: the value of the register that needs to be written, boolean type data.

Return: None

Example:

```
write_output_boolean_register(3,True)
```

5.44 write_output_float_register command

```
write_output_float_register(address, value)
```

Function: This command is used to write floating-point numbers to an output register which is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:47];
value: The value that needs to be written to the register. float type data.

Return: None

Example:

```
write_output_float_register(3,37.68)
```

5.45 write_output_integer_register command

```
write_output_integer_register(address, value)
```

Function: This command is used to write integers to an output register which is stored in the system memory space.

Parameter: address: Register address. Integer type data: [0:47];
value: The value that needs to be written to the register. integer type data.

Return: None

Example: `write_output_integer_register(3,37)`

Chapter 6 Modbus Command

6.1 Modbus

6.1.1 modbus_add_signal command

```
modbus_add_signal(IP, slave_number, signal_address, signal_type, signal_name  
, is_sequential = False, registers_count = 1, port = 502)
```

Function: This command is used to add a MODBUS TCP signal to the controller, which will be continuously monitored and controlled in the controller. In the current version, the modbus can add up to 200 signals. For output signals, before first invoking `modbus_set_output_signal/register()`, the MODBUS function code 1/3 will be used to continuously read values, and the MODBUS function code 5/6/15/16 will be used to continuously write values after calling. **Table 6-1** shows the use of function code in different situations.

Parameter: `IP`: Used to specify the IP address of Modbus appliance to which modbus signal is connected. string type data;
`slave_number`: Normally set to 255, can be freely selected between [0: 247], when set to 0, it is broadcast. When it is set to between [248: 254], it will be set to 255. Integer type data: [0: 247, 255]
`signal_address`: Specifies the address of the coil or register that this new signal should reflect. For this information, consult the configuration of the modbus appliance. Integer type data: [0:65535];
`signal_type`: Uses for specifying the type of signal to add. 0=digital input, 1=digital output, 2=register input, 3=register output. Integer type data: [0:3];
`signal_name`: Dedicated to identifying signals. If the string same as the added signal is provided, the new signal replaces the old signal. String type data;
`is_sequential`: an optional parameter used to check whether to update in order. If true, the signals will be updated in the order in which they were added. bool type data;
`registers_count`: an optional parameter meaning the number of consecutive read and write coils and registers of the signal. Integer type data: [1: 123].
`port`: an optional parameter representing the port of the slave station, which defaults to 502. Integer type data.

Return: None

Example:

```
modbus_add_signal("192.168.1.100", 255, 0, 3, "output1", False, 10, 502)
```

Table 6-1. Function Codes for Each Signal Type

Signal Type	Read		Write	
	Single Coil	Multi-coil	Single register	Multi-register
0 = Digital Input	2	2	/	/
1 = Digital Output	1	1	5	15
2 = Register Input	4	4	/	/
3 = Register Output	3	3	6	16

6.1.2 modbus_delete_signal command

```
modbus_delete_signal(signal_name)
```

Function: This command is used to delete signals identified by the given signal name.

Parameter: signal_name: The signal name, it's same as the signal name that should be deleted. String type data.

Return: None

Example: `modbus_delete_signal("output1")`

6.1.3 modbus_add_rw_signal

```
modbus_add_rw_signal(IP, slave_number, read_address, read_register_count,
    write_address, write_register_count, signal_name, is_sequential = False, port
    = 502)
```

- Function:** This command is used to add a MODBUSTCP signal to the controller and the function code is 23. The addresses for reading and writing can overlap. Before first calling the `modbus_set_output_register()` command, the function code used is 3, and becomes 23 after calling.
- Parameter:** `IP`: to specify the IP address of the modbus device to which the modbus signal is connected. string type data;
`slave_number`: freely selected between [0:247]; usually set as 255. When being set as 0, it is broadcast; when being set between [248: 254], it is sometimes set as 255. Integer type data: [0: 247, 255];
`read_address`: the starting address of the register to read. Integer type data;
`read_register_count`: the number of registers to read. Integer type data: [1: 123];
`write_address`: the starting address of the register to be written. Integer type data;
`write_register_count`: the number of registers to be written. Integer type data;
`signal_name`: specifically used for identifying signals. If a string identical to an already added signal is provided, the new signal will replace the old one. String type data;
`is_sequential`: an optional parameter used to check if the updating is updated in order. If it is true, the system will update signals according to the order of adding signals. Bool type data.
`port`: an optional parameter representing the port of the slave station, which defaults to 502. Integer type data.
- Return:** None
- Example:**

```
modbus_add_rw_signal("172.140.17.11", 255, 0, 10, 2, 10, "output1", False, 502)
```

6.1.4 modbus_get_signal_status command

```
modbus_get_signal_status(signal_name, is_secondary_program = False)
```


- Function:** This command is used to read the current value of a specific signal.
- Parameter:** `signal_name`: The name of the signal, the same as the name of the signal that should obtain the value. string type data.
- Return:** For a single digital signal: True or False, boolean type data;
For a single register signal: register values displayed as unsigned integers, integer type data;
For multiple digital signals: list of digital signals read, list type data;
For multiple register signals: list of register signals read, list type data.
- Example:** `modbus_get_signal_status("output1")`
Return value: True, False or an unsigned integer

6.1.5 modbus_send_custom_command

```
modbus_send_custom_command(IP, slave_number, function_code, data, oneshot = True  
    , port = 502)
```

- Function:** This command is used to send a command specified by the user to Modbus unit located on the specified IP address. Since the response will not be received, it cannot be used to request data. The user is responsible for supplying data which is meaningful to the supplied function code. The built-in function takes care of constructing Modbus frame, so the user should not be concerned with the length of the command.
- Parameter:** IP: to specify the IP address of Modbus appliance that custom command should send to. string type data;
slave_number: slave or server address. Integer type data: [0: 255];
function_code: function code used to specify custom commands. integer type data;
data: a list of integers. Each entry must be a valid byte [0:255] value.
list type data.
oneshot: to check whether to send requests only once. Boolean type data. This is an optional parameter, and it defaults to be True.
port: an optional parameter representing the port of the slave station, which defaults to 502. Integer type data.

Return: None

Example: `modbus_send_custom_command ("172.140.17.11", 103, 6,[17,32,2,88], True, 502)`
Description: The watchdog timeout on the Backhoff BK9050 is set to 600ms.
This is done as follows: use modbus function code 6 (preset a single register), then provide the register address in the first two bytes of the data array ([17,32]=[0x1120]), and provide the required register contents in the last two bytes ([2,88]=[0x0258]=dec 600).

6.1.6 modbus_set_output_register command

```
modbus_set_output_register(signal_name, register_value)
```

Function: This command is used to set the output register signal identified by the given name to the given value.

Parameter: `signal_name`: The signal name that identifies the output register signal that has been added in advance. string type data;
`register_value`: The register value to be written. If it is an integer value, then it ranges from 0 to 65535. If it exceeds the range, it will be automatically forced to be within the range according to memory type conversion or memory truncation. If there is a list, then the minimum list length should be 1 and less than or equal to the number of registers declared when the signal was added. If it is a list with smaller length, then the register with the higher MODBUS address will retain its original value. integer or list type data.

Return: None

Example:

```
modbus_set_output_register("output1",300)
Parameters:
signal_name="output1"
register_value=300
```

6.1.7 modbus_set_output_signal command

```
modbus_set_output_signal(signal_name, digital_value)
```

Function: This command is used to set the output digital signal identified by a given name to a given value.

Parameter: `signal_name`: The signal name that identifies the output digital signal that has been added in advance. string type data;
`digital_value`: the coil value to be written and it should be Boolean or Boolean list type. If it is a list, then its minimum length is 1 and its length must be less than or equal to the number of coils declared when adding the signal. If it is a list with a smaller length, then coils with higher MODBUS addresses will retain their original values. boolean or list type data.

Return: None

Example:

```
modbus_set_output_signal("output2",True)
```

6.1.8 modbus_set_runstate_dependent_choice command

```
modbus_set_runstate_dependent_choice(signal_name, runstate_choice)
```

Function: This command is used to set whether the output signal must remain in the state from the program, or whether the program is not running high or low.

Parameter: `signal_name`: The signal name that identifies the output digital signal that has been added in advance. string type data;
`runstate_choice`: 0 = Maintain program state; 1 = Set low when the program is not running; 2 = Set high when the program is not running; 3 = Output low when the program stops, Output high when the program is running. Integer type data: [0: 3].

Return: None

Example: `modbus_set_runstate_dependent_choice("output2",1)`

6.1.9 modbus_set_signal_update_frequency command

```
modbus_set_signal_update_frequency(signal_name, update_frequency)
```

Function: This command is used to set the frequency at which the robot sends a request to the Modbus controller to read or write a signal value.

Parameter: `signal_name`: The signal name that identifies the output digital signal that has been added in advance. string type data;
`update_frequency`: Used to specify the update frequency, in Hz. Integer type data: [0: 250].

Return: None

Example: `modbus_set_signal_update_frequency("output2",20)`

6.1.10 modbus_set_signal_watchdog

```
modbus_set_signal_watchdog(signal_name, timeout)
```

Function: This command is used to set the watchdog time of the modbus valid signal, that is, within the watchdog time after a signal error occurs, reading and writing the signal by using a script will not throw an exception. Users can use the two commands `modus_get_error()` and `modus_get_time_since_signal_invalid()` to check whether getting the signal is abnormal before reading or writing. If not set, the default watchdog time is 5 seconds.

Parameter: `signal_name`: signal name. String type;
`timeout`: watchdog timeout time(unit: s), ranging from 0 to 300. Float type.

Return: None

Example: `modus_set_signal_watchdog("MODBUS", 120)`

6.1.11 read_port_bit command

```
read_port_bit(address)
```

Function: This command is used to read the address value of the slave digital (coil) on the Modbus.

Parameter: `address`: The address of the port (see Port Mapper on the Using Modbus Server page of the support website). Integer type data: [0:127].

Return: Port saved values, boolean type data.

Example: `boolval=read_port_bit(3)`
Return value: True or False

6.1.12 read_port_register command

```
read_port_register(address, is_signed=False)
```

- Function:** This command is used to read the address values from the slave register on the Modbus.
- Parameter:** address: the address of the port, integer type data.
is_signed: Whether the data is signed, default is False, optional parameter, boolean type data.
- Return:** Integer type data, the port holds a signed integer value [-32768:32767] or [0:65535].
- Example:** `intval=read_port_register(256,True)`
Return value: A signed integer

6.1.13 write_port_bit command

```
write_port_bit(address, value)
```

- Function:** This command is used to write the address values in the slave digital (coils) on the Modbus.
- Parameter:** address: The address of the port (see Port Mapper on the "Using Modbus Server" page of the support website) integer type data;
value: The value to be set in the register. boolean type data.
- Return:** None
- Example:** `write_port_bit(3,True)`

6.1.14 write_port_register command

```
write_port_register(address, value)
```

- Function:** This command is used to write the address values in the slave register on the Modbus.
- Parameter:** address: The address of the port. integer type data;
value: The value to be set in the port (0:65536) or (-32768:32767). integer type data.
- Return:** None
- Example:** `write_port_register(3,100)`

6.1.15 modbus_get_error

```
modbus_get_error(signal_name, error_source = 0)
```

Function: This command is used to get the current error code of the signal.

Parameter: signal_name: signal name, same as the signal that should obtain the value. String type data;
error_source: source of the error code. Integer type data: 0 = composite,, 1 = device, 2 = connection. (an optional parameter, its default value is 0).

Return: Error code, list type data. Device error codes refers to **Table 6-2**. The value of connection error code ranges from 1 to 113, and more information sees [Wikipedia related pages](#)□[TCP/IP errors introduction from IBM official website](#)□

Example:

```
ec = modbus_get_error("MODBUS_1")
```

Table 6-2. Device Error Codes

Value	Description
-2	Not connected
-1	Being connected
0	No error
1	Illegal function code = ILLEGAL FUNCTION CODE
2	Illegal data address = ILLEGAL DATA ADDRESS
3	Request for including an illegal data value = ILLEGAL DATA VALUE
4	Slave or server failure = SLAVE OR SERVER FAILURE
5	Master request has been received, but more time is needed to complete the requested operation = ACKNOWLEDGE
6	Slave or server is busy = SLAVE OR SERVER BUSY
7	Slave station refuses to process the request from the host = NEGATIVE ACKNOWLEDGE
8	Memory parity error = MEMORY PARITY
9	Undefined abnormality = NOT DEFINED
10	Gateway path unavailable = GATEWAY PROBLEM
11	Gateway target device failed to respond = GATEWAY TARGET FAILURE

NOTICE

1. If the error source is composite, a list with two lengths will be returned. The first element of the list is the device error code and the second is the connection error code.
2. If the error source is a device or connection, then a list with just one length will be returned, and the data content is the corresponding error code.

6.1.16 modbus_get_time_since_signal_invalid


```
modbus_get_time_since_signal_invalid(signal_name)
```

Function: For invalid signals (such as device or communication errors), this command is used to obtain the time interval value (unit:seconds) between the invalid signal and the last successful reading and writing operation. For valid signals, 0 is returned.

Parameter: signal_name: signal name, identical to the signal that should obtain the value. string type data.

Return: the time since the last successful reading and writing operation. float type data.

Example: `dleta = modbus_get_time_since_signal_invalid("MODBUS_1")`

6.1.17 modbus_reset_connection

```
modbus_reset_connection(connection_id, is_blocking = True, port = 502)
```

Function: This command is used to reset connection.

Parameter: connection_id: IP address of MODBUS device. String type data.
is_blocking: to check if the execution is blocked. boolean type data (optional parameter, and it defaults to be True).
port: an optional parameter representing the port of the slave station, which defaults to 502. Integer type data.

Return: None

Example: `modbus_reset_connection("192.168.1.123", True, 502)`

Chapter 7 Communication Command

7.1 SOCKET

7.1.1 socket_open command

```
socket_open(address, port, socket_name="socket_0")
```

Function: This command is used to establish TCP/IP network communication, and the connection timeout is 2 seconds.

Parameter: address: server address, string type data;
port: port number, integer type data;
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: Boolean type data. First call: if the connection is successful, return True; if it fails, return False. Subsequent calls: if the connection is active, return True; if it is disconnected, the current socket will be closed and reconnected. If the reconnection is successful, return True; if it fails, return False.

Example: `socket_open("192.168.1.119", 23333, "socket_1")`
Return value: True or False

7.1.2 socket_close command

```
socket_close(socket_name="socket_0")
```

Function: This command is used to close TCP/IP network communication.

Parameter: socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: None

Example: `socket_close("socket_3")`

7.1.3 socket_is_connected command

```
socket_is_connected(socket_name = "socket_0")
```

Function: This command is used to get the communication status of the TCP/IP network to determine whether it is connected to the server.

Parameter: socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: boolean type data, returns True if connection succeeded, returns False if the connection failed.

Example: `socket_is_connected(socket_name = "socket_3")`
Return value: True or False

7.1.4 socket_get_var command

```
socket_get_var(var_name, socket_name="socket_0")
```

Function: This command is used to read an integer data from the server; The interface will send a "GET <var_name>\n" message to the specified server, and wait for the server to reply with content in the format "<var_name><int>\n". The timeout period for waiting for a reply is 2 seconds. If the reply format does not meet the protocol requirements or the timeout occurs, then return is 0.

Parameter: var_name: variable name, string type data;
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: Integer type data, the integer value returned by the server. if the returned value is 0, there may be an error occurred (unless the server itself is 0). The return value range is [-2147483648, 2147483647], beyond which the maximum or minimum value will be returned.

Example: `joint_0_pos=socket_get_var("JOINT_0")`
Description: Sends the string "GET JOINT_0\n" to the server socket_0 and waits for the server to reply with something in the format "JOINT_0 1000\n" with a waiting timeout of 2 seconds
Return value: The integer value obtained from the server

7.1.5 socket_read_ascii_float command

```
socket_read_ascii_float(number, socket_name="socket_0", timeout=2)
```

Function: This command is used to read multiple float type data in ASCII format from a connected TCP/IP server;

If the data transmitted by the server needs to be enclosed in parentheses, and the data is separated by commas "," or spaces " ", similar to "(3.14, 2.562, 8.24, 3.4434)";

If the data transmitted by the server is in a format similar to "(3.14, „2.562, „8.24, 3.4434)", and the four data are still read correctly.

Parameter: number: the number of data to be read, integer type data: [1:30];
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter);
timeout: The unit is seconds. If the value is less than or equal to 0, then wait timeout is infinite, float type data (optional parameter).

Return: A list data is returned, where the first element is length of successfully recieved data and the remaining elements are float type data read.
If the read fails, the first element of the list is 0 and the other elements are nan, similar to [0, nan, nan];
If the read is successful, a list of data is returned, similar to [3, 1.444, 2.234, 3.444];
If the first element of the list is not equal to the parameter <number>. the returned data list includes the data length and data that were successfully read, then the unread data will be nan, like [2, 1.444, 2.234, nan].

Example: list_of_three_floats=socket_read_ascii_float(3)
Parameters:
number=3
socket_name=socket_0, the default name
timeout=2, the default timeout
Return value: The data list read, [3, 3.12, 3.11, 8.24], [2, 3.12, 3.11,nan] or [0, nan, nan, nan]

7.1.6 socket_read_binary_integer command

```
socket_read_binary_integer(number, socket_name="socket_0", timeout=2)
```

- Function:** This command is used to read multiple 32-bit integer data from a connected TCP/IP server in the form of network byte order.
- Parameter:** number: the number of data to be read, integer type data: [1:30];
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter);
timeout: The unit is seconds. If the value is less than or equal to 0, then wait timeout is infinite, float type data (optional parameter).
- Return:** List type data, the list of data read, the first element of the list is the number of data successfully read, the second element begins as the integer data read;
If the read fails, the first element of the list is 0 and the other elements will be -1, like [0, -1, -1, -1];
If the read is successful, a list of data is returned, like [2, 100, 200];
If the length of received data is not equal to the parameter <number>, the returned data list includes the data length and data that were successfully read, and the unread data will be -1, like [2, 100, 200, -1].
- Example:**
- ```
list_of_three_ints=socket_read_binary_integer(3)
Parameters:
number=3
socket_name=socket_0, the default name
timeout=2, the default timeout
Return value: The list of data read, [3, 100, 200, 300] or [0, -1, -1, -1] or [2, 100, 200, -1]
```

### 7.1.7 socket\_read\_byte\_list command

```
socket_read_byte_list(number, socket_name="socket_0", timeout=2)
```

- Function:** This command is used to read multiple bytes of data from a connected TCP/IP server.
- Parameter:** number: the number of data to be read, integer type data: [1:512];  
socket\_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket\_0", string type data (optional parameter);  
timeout: The unit is seconds. If the value is less than or equal to 0, then wait timeout is infinite, float type data (optional parameter).
- Return:** List type data, the list of data read, the first element of the list is the number of successfully read data, the second element begins as the read byte data;  
If the read fails, the first element of the list is 0 and the other elements are -1, like [0, -1, -1, -1];  
If the read is successful, a list of data is returned, similar to [2, 100, 200];  
If the length of received data is not equal to the parameter <number>, and returned data list includes the data length and data that were successfully read, then the unread data is -1,[2, 100, 200, -1].
- Example:**
- ```
list_of_three_byts=socket_read_byte_list(3)
Parameters:
number=3
socket_name=socket_0, the default name
timeout=2, the default timeout
Return value: The data list read, [3, 100, 200, 123] or [0, -1, -1, -1] or [2, 100, 200, -1]
```

7.1.8 socket_send_byte_list command

```
socket_send_byte_list(value, socket_name="socket_0")
```

Function: This command is used to send a set of byte data to a connected TCP/IP server.

Parameter: value: the byte data to be sent, a list of integer, list type data;
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter);

Return: Integer type data, if it is successful to send the byte data, the sent byte data is returned.

Example:

```
socket_send_byte_list([1, 0x3, 0x2E, 3])
```

Parameters:
value=[1, 0x3, 0x2E, 3]
socket_name=socket_0, the default name
Return value: If the byte data is sent out successfully, it returns to 4

7.1.9 socket_read_string command

```
socket_read_string(socket_name="socket_0", prefix="", suffix="", timeout=2)
```


Function: This command is used to read all data from the socket according to the rules of <prefix> and <suffix> settings and return the string data.

Parameter: socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter);
prefix: string matching prefix, string type data (optional parameter);
suffix: string matching suffix, string type data (optional parameter);
timeout: The unit is seconds. If the value is less than or equal to 0, then wait timeout is infinite, float type data (optional parameter).

Return: String type data, read string variables. When prefix and suffix are not specified, all string data read will be returned; when only prefix is specified, all read string data after prefix will be returned except the prefix string itself; when only suffix is specified, all read string data before suffix will be returned except the suffix string itself; when both prefix and suffix are specified, the function will return the string between prefix and suffix, excluding the prefix and suffix strings themselves. The data that has not been read this time will remain in the socket and can be read cyclically. When the read timeout, the function will return an empty string.

Example:

```
while(True):
    string_from_server=socket_read_string(prefix="<",suffix=">")
    if len(string_from_server): ==0
        break
    txtmsg(string_from_server)
```

Description: Suppose the string sent socket_0 server is
"<111><222><333><444>"
Return value: returns "111", "222", "333", "444"

Note: Socket_read_string(suffix="\n") script can be used to implement a row-by-row reading of string data from the server. The command have a maximum amount of data obtained in a single pass of 2047.

7.1.10 socket_send_byte command

```
socket_send_byte(value, socket_name="socket_0")
```

- Function:** This command is used to send a byte of data to a connected TCP/IP server and can be used to send ASCII character data: for example, 10 stands for "\n".
- Parameter:** value: The byte data that needs to be sent, string type data or integer type data. For string type data the length must not exceed 1; for integers, the number must be in range [0-255]. If the integer value exceed the range, it will be automatically converted to the range in the way of memory type conversion or memory truncation;
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).
- Return:** Boolean type data, returns True if sent successfully, returns False if sent fails.
- Example:**
- ```
socket_send_byte(10)
parameter:
value=10, ASCII character data, 10 represents"\n"
socket_name=socket_0, default name
Return value:True or False
socket_send_byte("a")
parameter:
value="a", string character data
socket_name=socket_0, default name
Return value:True or False
```

### 7.1.11 socket\_send\_int command

```
socket_send_int(value, socket_name="socket_0")
```

- Function:** This command is used to send integer (int32\_t) data to a connected TCP/IP server in the form of network byte order.
- Parameter:** value: the byte data that needs to be sent, integer type data;  
socket\_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket\_0", string type data (optional parameter).
- Return:** Boolean type data, returns True if sent successfully, returns False if sent fails.
- Example:**
- ```
socket_send_int(10)
Return value:True or False
```

7.1.12 socket_send_line command

```
socket_send_line(str, linefeed_type=0, socket_name="socket_0")
```

Function: This command is used to send string data in ASCII encoding to the connected TCP/IP server. When sending, a new line of the user-specified character will be added at the end of the string data <str>.

Parameter: str: string to send, string type data;
linefeed_type: type of line break, int, 0: the character "\n" will be added at the end of the string data, 1: the character "\r" will be added at the end of the string data, 2: the character "\r\n" will be added at the end of the string data, the default type is 0 (optional parameter);
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: Boolean type data, if sent successfully, return True, if sent failed, return False.

Example: `socket_send_line("Hello world")`
Description: Send "Hello world\n" to the server socket_0
Return value: True or False

7.1.13 socket_send_string command

```
socket_send_string(str, socket_name="socket_0")
```

Function: This command is used to send string data in ASCII encoding to a connected TCP/IP server.

Parameter: str: Support Python built-in data types: string, list, boolean, integer, float and other data types;
socket_name: name of socket, when the name is not specified or the specified name is empty, use the default name "socket_0", string type data (optional parameter).

Return: Boolean type data, if sent successfully, return True, if the send failed, return False.

Example: `socket_send_string("Hello world")`
Description: Send "Hello world" to the server socket_0
Return value: True or False

7.1.14 socket_set_var command

```
socket_set_var(name, value, socket_name="socket_0")
```

Function: This command is used to send the data with the content of "SET <name> <value>\n" to the connected TCP/IP server, to realize the function of setting the value of the server integer variable.

Parameter: name: variable name, string type data;
value: variable value, integer type data;
socket_name: name of socket, when the name is not specified or the specified name is empty, the default name "socket_0", string type data (optional parameter) is used.

Return: None

Example: `socket_set_var("JOINT_0",1000)`
Description: Sends the string "SET JOINT_0 1000\n" to the server socket_0

7.2 RPC

7.2.1 rpc_factory command

```
rpc_factory(type, url)
```

Function: This command is used to create a remote call (RPC) handle for implementing remote call functionality.

Parameter: type: RPC type, currently only support "xmlrpc" type, string type data;
url: RPC server address, the format is "<ip-address>:<port>" or "http://<ip address>:<port>/<path>", such as "127.0.0.1:5050" or "http://127.0.0.1:5050/RPC2", string type data.

Return: Object, RPC handle.

Example: `camera=rpc_factory("xmlrpc","http://127.0.0.1:5050/RPC2")`
`camera.capture()`
Description: Create an RPC remote call handle for the camera and call the camera's methods to take pictures
Return value: RPC handle

Chapter 8 Serial Communication

This chapter mainly introduces those commands involving MODBUS-RTU and RS-485 serial communication.

8.1 RS-485

This section mainly describes those commands involving how to view, read and write the RS-485 communication data of the robot tool terminal or the controller.

8.1.1 tool_serial_mode command

```
tool_serial_mode()
```

Function: This command is used to view which mode the robot tool terminal RS-485 communication is in.

Parameter: None

Return: String type data, Modbus-RTU" or "RS485", "CLOSE" if neither is enabled.

Example:

```
tool_serial_mode()  
Return value:"RS-485"
```

8.1.2 masterboard_serial_config command

```
masterboard_serial_config(enable, baud = 115200, parity = 1, stop_bit = 1, size  
= 8, modbus_rtu = False)
```

- Function:** This command is used to enable/disable the RS-485 communication function of the robot controller, and to configure the serial port.
- Parameter:** enable: the communication of the serial port is enabled or disabled, boolean type data.
baud: integer type data: 2400, 4800, 9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600, 500000. When not specified, the parameter is 115200 by default.
parity_bit: serial port parity, integer type data: 0 (no parity), 1 (odd parity), and 2 (even parity). When not specified, it is 1 (odd parity) by default.
stop_bit: serial port stop bit, integer type data: 1, 2.
size: the bit width of the serial port. When not specified, it is 8 by default.
integer type data: 7 or 8 (optional parameter). if it is in the MODBUS-RTU mode, the size must be 8.
modbus_rtu: whether to enable MODBUS-RTU mode, boolean type data. True = MODBUS-RTU mode enabled; False = RS485 mode enabled. The default value of False is used when not specified.
- Return:** boolean type data. True = Configuration succeeded; False = Configuration failed.
- Example:** `masterboard_serial_config(True, 115200, 1, 1, 8, False)`
Return Value: True or False

8.1.3 masterboard_serial_read command

```
masterboard_serial_read(read_count, is_number = True, timeout = 0.1)
```

- Function:** This command is used to read the RS-485 communication data from the robot controller.
- Parameter:** `read_count`: to read data length, integer type data: greater than or equal to 0.
`is_number`: Whether to return byte data in integer format. If not specified, it is True by default, boolean type data (optional parameter).
`timeout`: timeout time in seconds. If less than or equal to 0, the wait timeout is infinite. When not specified, it is 0.1 by default. float type data (optional parameter).
- Return:** Bytearray or list. If `is_number` is True, the return value is a list of type integer. If False, the return value is bytes array. If a timeout occurs, all data that has been read is returned.
- Example:**
- ```
masterboard_serial_read(4, True, 0.1)
Parameters:
read_count=4
is_number=True (the default value)
time_out=0.1 (the default value)
Return value: list of read data,[12, 99, 255, 1]
```

### 8.1.4 masterboard\_serial\_write command

```
masterboard_serial_write(data)
```

- Function:** This command is used to send out the data via the robot controller.
- Parameter:** `data`: data to be sent, string type data, integer type data, integer list type data, bytes type data or bytearray type data. If the value exceeds the range of [0-255], it will be automatically cast into the range according to the way of memory type conversion or memory truncation.
- Return:** Boolean type data. True = send successfully; False = send failed.
- Example:**
- ```
masterboard_serial_write([123,56,0])
Parameters:
data=[123,56,0]
Return Value: True or False
```

8.1.5 masterboard_serial_mode command

```
masterboard_serial_mode()
```

Function: This command is used to view which mode the robot controller RS-485 communication is in.

Parameter: None

Return: integer type data. 0 = serial port off; 1 = RS485 mode; 2 = MODBUS-RTU mode.

Example:

```
mode=masterboard_serial_mode()  
Return value: 2
```

8.1.6 masterboard_serial_flush command

```
masterboard_serial_flush()
```

Function: The command is used to refresh the RS-485 data from the controller.

Parameter: None

Return: None

Example:

```
masterboard_serial_flush()
```

8.2 MODBUS-RTU

This section describes some commands used to read and write the MODBUS-RTU slave via the tool terminal or the controller.

8.2.1 tool_modbus_read_bits command

```
tool_modbus_read_bits(slave, addr, len, time_out=5)
```


Function: This command is used to read the Modbus-RTU slave coils via the robot tool terminal RS- 485 communication, the MODBUS function code is 0x01.

Parameter: slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
len: to read the length of the data, the range is 1 - 125, when the len is greater than 1, the data will be read down sequentially from the slave's addr, integer type data;
time_out: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.

Return: List type data, read data list.

Example: read_data=tool_modbus_read_bits (1, 1, 1, 0)
Return value: the coil value read;

8.2.2 tool_modbus_read_input_bits command

```
tool_modbus_read_input_bits(slave, addr, len, timeout=5)
```

Function: This command is used to read the Modbus-RTU slave coils via the robot tool terminal RS- 485 communication, the MODBUS function code is 0x02.

Parameter: slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
len: to read the length of the data, the range is 1 - 125, when the len is greater than 1, the data will be read down sequentially from the slave's addr, integer type data;
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.

Return: List type data, data list that has been read.

Example: read_data=tool_modbus_read_input_bits (1, 1, 1, 0)
Return value: the value read from the coil 1 of the slave 1

8.2.3 tool_modbus_read_registers command

```
tool_modbus_read_registers(slave, addr, len, time_out=5)
```

- Function:** This command is used to read the MODBUS-RTU slave holding registers via the robot tool terminal RS-485 communication, the MODBUS function code is 0x03.
- Parameter:** slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
len: to read the length of the data, the range is 1 - 125, when the len is greater than 1, the data will be read down sequentially from the slave's addr, integer type data;
time_out: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** List type data, read data list.
- Example:** `read_data=tool_modbus_read_registers (1, 1, 1, 0)`
Return value: The register value read;

8.2.4 tool_modbus_read_input_registers command

```
tool_modbus_read_input_registers(slave, addr, len, timeout=5)
```

- Function:** This command is used to read the Modbus-RTU slave registers via the robot tool terminal RS-485 communication, the MODBUS function code is 0x04.
- Parameter:** slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
len: to read the length of the data, the range is 1 - 125, when the len is greater than 1, the data will be read down sequentially from the slave's addr, integer type data;
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** List type data, data list that has been read.
- Example:** `read_data=tool_modbus_read_input_registers (1, 1, 1, 0)`
Return value: the value read from the register 1 of the slave 1

8.2.5 tool_modbus_write_single_bit command

```
tool_modbus_write_single_bit(slave, addr,data, timeout=5)
```

- Function:** This command is used to read a single Modbus-RTU slave coil via the robot tool terminal RS-485 communication, the MODBUS function code is 0x05.
- Parameter:** slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
data: The data to be written, integer type data;
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** Integer type data, the data volume that has been written successfully, i.e. 1, if it is less or equal to 0, it indicates a writing failure.
- Example:**

```
read_data=tool_modbus_write_single_bit (1, 1, 1, 0)
Return value:1
```

8.2.6 tool_modbus_write_bits command

```
tool_modbus_write_bits(slave, address, data, timeout=5)
```

- Function:** This command is used to write the multiple Modbus-RTU slave coils via the robot tool terminal RS-485 communication, the MODBUS function code is 0x0F.
- Parameter:** slave: slave, range 1 - 247, integer type data;
address: address, the range is greater than or equal to 0, integer type data;
data: The data to be written, Boolean, integer (0 will be implicitly converted to False and the nonzero elements will be implicitly converted to True), list type data (each element of the list must be integer or boolean). When the data is list, it is written sequentially down to the slave's addr address;
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** Boolean type data, write successfully return True, otherwise False.
- Example:**

```
tool_modbus_write_bits (1, 1, 1)
Return value:True;
```

8.2.7 tool_modbus_write_single_register command

```
tool_modbus_write_single_register(slave, addr,data, timeout=5)
```

- Function:** This command is used to read a single Modbus-RTU slave register via the robot tool terminal RS-485 communication, the MODBUS function code is 0x06.
- Parameter:** slave: slave, range 1 - 247, integer type data;
addr: address, the range is greater than or equal to 0, integer type data;
data: The data to be written, integer type data;
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** Integer type data, the data volume to be read successfully, i.e. 1, if it is less or equal to 0, it indicates a writing failure.
- Example:**
- ```
read_data=tool_modbus_write_single_register (1, 1, 1)
Return value:1
```

### 8.2.8 tool\_modbus\_write\_registers command

```
tool_modbus_write_registers(slave, address, data, timeout=5)
```

- Function:** This command is used to write the multiple Modbus-RTU slave holding registers via the robot tool terminal RS-485 communication, the MODBUS function code is 0x10.
- Parameter:** slave: slave, range 1 - 247, integer type data;  
address: address, the range is greater than or equal to 0, integer type data;  
data: The data to be written, integer [0, 65535], list type data (each element of the list must be integer [0, 65535]). When the data is list, it is written sequentially down to the slave's addr address;  
timeout: timeout time, unit: seconds, if less than or equal to 0, the waiting timeout time is infinitely long, float type data.
- Return:** Boolean type data, write successfully return True, otherwise False.
- Example:**
- ```
tool_modbus_write_registers (1, 1, 1)
Return value:True;
```

8.2.9 masterboard_modbus_rtu_read_bits command

```
masterboard_modbus_rtu_read_bits(slave, address, read_count, timeout = 3.0)
```

Function: This command is used to read the MODBUS-RTU slave coils via the robot controller RS-485 communication, the MODBUS function code is 0x01.

Parameter: slave: slave, range 1 - 247, integer type data.
address: address, the range is greater than or equal to 0, integer type data.
read_count: length of data to be read, range 1-125, integer type data. If it is greater than 1, it will start reading from the slave's addr and work its way down.
timeout: timeout time in seconds. If less than or equal to 0, the waiting timeout time is infinitely long, default to be 5s, float type data (optional parameter);

Return: list type data, a list of data to be read.

Example: `masterboard_modbus_rtu_read_bits(1, 0, 2, 1)`
Return value: [0, 0]

8.2.10 masterboard_modbus_rtu_read_input_bits command

```
masterboard_modbus_rtu_read_input_bits(slave, address, read_count, timeout = 3.0)
```

Function: This command is used to read the MODBUS-RTU slave input coils via the robot controller RS-485 communication, the MODBUS function code is 0x02.

Parameter: slave: slave, range 1-247, integer type data.
address: address, greater than or equal to 0, integer type data.
read_count: length of data to be read, range 1-125, integer type data. If it is greater than 1, it will start reading from the slave's addr and work its way down.
timeout: timeout time in second. If less than or equal to 0, the waiting time is infinite, default is 5 seconds, float type data (optional parameter).

Return: list type data, a list of data to be read.

Example: `read_data=masterboard_modbus_rtu_read_input_bits(1, 1, 1, 0)`
Return value: the value read from the coil 1 of the slave 1

8.2.11 masterboard_modbus_rtu_read_registers command

```
masterboard_modbus_rtu_read_registers(slave, address, read_count, timeout = 3.0)
```

Function: This command is used to read the MODBUS-RTU slave holding registers via the robot controller RS-485 communication, the MODBUS function code is 0x03.

Parameter: slave: slave, range 1-247, integer type data.
address: address, greater than or equal to 0, integer type data.
read_count: length of data to be read, range 1-125, integer type data. If greater than 1, it will start reading from the slave's addr and work its way down.
timeout: timeout time in seconds. If less than or equal to 0, the wait timeout is infinite, default is 5 seconds, float type data (optional parameter).

Return: list type data, a list of data to be read.

Example: `masterboard_modbus_rtu_read_registers(1, 0, 2, 1)`
Return value: [0, 0]

8.2.12 masterboard_modbus_rtu_read_input_registers command

```
masterboard_modbus_rtu_read_input_registers(slave, address, read_count, timeout = 3.0)
```

Function: This command is used to read the MODBUS-RTU slave registers via the robot controller RS-485 communication, the MODBUS function code is 0x04.

Parameter: slave: slave, range 1 - 247, integer type data.
address: address, greater than or equal to 0, integer type data.
read_count: length of data to be read, range 1-125. If greater than 1, the data will be read from the slave's addr, and the integer data will be read from the slave's addr.
timeout: timeout time in seconds. If less than or equal to 0, the wait time is infinite, default to be 5s, float type data (optional parameter).

Return: list type data, a list of data to be read.

Example: `read_data=masterboard_modbus_rtu_read_input_registers(1, 1, 1, 0)`
Return value: the value read from the register 1 of the slave 1

8.2.13 masterboard_modbus_rtu_write_single_bit command

```
masterboard_modbus_rtu_write_single_bit(slave, address, data, timeout = 3)
```

Function: This command is used to write a single MODBUS-RTU slave coil via the robot controller RS-485 communication, the MODBUS function code is 0x05.

Parameter: slave: slave, range 1 - 247, integer type data.
address: address, greater than or equal to 0, integer type data.
data: the data to be written, integer type data.
timeout: timeout time, unit: seconds. If less than or equal to 0, the waiting timeout time is infinitely long, default to be 5s, float type data (optional parameter).

Return: Integer type data, the data volume that has been written successfully, i.e. 1. If it is less or equal to 0, it indicates a write failure.

Example: `read_data=masterboard_modbus_rtu_write_single_bit(1, 1, 1, 3)`
Return value: 1

8.2.14 masterboard_modbus_rtu_write_bits command

```
masterboard_modbus_rtu_write_bits(slave, address, data, timeout = 3)
```

- Function:** This command is used to write the multiple MODBUS-RTU slave coils via the robot controller RS-485 communication, the MODBUS function code is 0x0F.
- Parameter:** slave: slave, range 1 - 247, integer type data.
address: address, greater than or equal to 0, integer type data.
data: the data to be written. Boolean, integer (0 will be implicitly converted to False and the nonzero elements will be implicitly converted to True), list type data (each element of the list must be integer or boolean). When the data is list, it is written sequentially down to the slave's address;
timeout: timeout time in seconds. If less than or equal to 0, the waiting timeout time is infinitely long, default to be 5s, float type data (optional parameter).
- Return:** Integer type data. The number of the coil is written successfully. If it is less or equal to 0, there is a failure in writing the number of the coil.
- Example:**

```
masterboard_modbus_rtu_write_bits(1, 0, [1], 3)
```


Return value: 1

8.2.15 masterboard_modbus_rtu_write_single_register command

```
masterboard_modbus_rtu_write_single_register(slave, address, data, timeout = 3)
```

- Function:** This command is used to write a single MODBUS-RTU slave register via the robot controller RS-485 communication, the MODBUS function code is 0x06.
- Parameter:** slave: slave, range 1 - 247, integer type data.
address: address, greater than or equal to 0, integer type data.
data: the data to be written, integer type data.
timeout: timeout time in seconds. If less than or equal to 0, the waiting timeout time is infinitely long, default to be 5s, float type data (optional parameter).
- Return:** Integer type data, the data volume that has been written successfully, i.e. 1. If it is less or equal to 0, it indicates a writing failure.
- Example:**

```
read_data=masterboard_modbus_rtu_write_single_register(1, 1, [1], 3)
```


Return value: 1

8.2.16 masterboard_modbus_rtu_write_registers command

```
masterboard_modbus_rtu_write_registers(slave, address, data, timeout = 3)
```

Function: This command is used to write the multiple MODBUS-RTU slave holding registers via the robot controller RS-485 communication, the MODBUS function code is 0x10.

Parameter: slave: slave, range 1 - 247, integer type data.

address: address, the range is greater than or equal to 0, integer type data.

data: the data to be written, integer [0, 65535], list type data (each element of the list must be integer [0, 65535]). When the data is list, it is written sequentially down to the slave's address.

timeout: timeout time in seconds. If less than or equal to 0, the waiting timeout time is infinitely long, default to be 5s, float type data (optional parameter).

Return: Integer type data, the number of the register is written successfully. If it is less or equal to 0, there is a failure in writing the number of the register.

Example:

```
masterboard_modbus_rtu_write_registers(1, 0, [12], 3)
```


Return value: 1

Chapter 9 System Command

9.1 Thread

9.1.1 start_thread command

```
start_thread(function, args=(), detach_mode=True)
```

Function: This command is used to start threads.

Parameter: function: the function that threads perform. callable type data.
arg: the argument passed to the function. tuple type data. This parameter is optional and defaults to '()', representing no parameters.
detach_mode: used to set the thread start mode. True: detached mode, False: non-detached mode. boolean type data. This parameter is optional and defaults to True.

Return: The ID of the newly created thread, which can be used as the parameter of stop_thread and join_thread functions to realize stopping thread and waiting for the thread to stop.

Example:

```
def thread_test(a, b):
    textmsg("Run thread: ", a+b)
    sleep(1)
    textmsg("Thread end.")
global thread_id
thread_id = start_thread(thread_test, (1, 2), False)
sleep(2)
Parameters:
function = thread_test -> to execute the 'thread\_test' function
as a thread;
args = (1, 2) -> pass arguments (1, 2) to 'thread_test' function;
detach_mode = False -> run in non-detach mode.
```

9.1.2 join_thread command

```
join_thread(thread_id)
```

Function: This command is used to wait for the thread with the thread ID of the given parameter to end.

Parameter: thread_id: ID of the thread returned by the starting thread.

Return: None

Example:

```
def thread_test(a, b):  
    textmsg("Run thread: ", a+b)  
    sleep(1)  
    textmsg("Thread end.")  
global thread_id  
thread_id = start_thread(thread_test, (1, 2), False)  
join_thread(thread_id)
```

9.1.3 stop_thread command

```
stop_thread(thread_id)
```

Function: This command is used to shut down threads; outside of threads to close threads.

Parameter: thread_id: The thread ID returned by the starting thread.

Return: None

Example:

```
id=start_thread(thread_test,(1,2))  
stop_thread(id)
```

9.2 Task execution

9.2.1 pause command

```
pause()
```

Function: This command is used to pause the task.

Parameter: None

Return: None

Example:

```
pause()  
Pause the task
```

9.2.2 halt command

```
halt()
```

Function: This command is used to stop the task.

Parameter: None

Return: None

Example: `halt()`
Stop the task

9.3 Prompts for a warning

9.3.1 textmsg command

```
textmsg(s1, s2='')
```

Function: This command is used to send the string connected by s1 and s2 to EliRobot and display it in the log column of EliRobot.

Parameter: s1: message string, can also send other types of variables, integer, boolean, list type data, etc.;
s2: message string, can also send other types of variables, integer, boolean, list type data, etc. (optional parameters, default is an empty string).

Return: None

Example: `textmsg("value=", 3)`

Note: Frequent and heavy calls to this script function are not recommended, as this may cause software performance issues.

9.3.2 popup command

```
popup(s, title='Popup', warning=False, error=False, blocking=False)
```

Function: This command is used for popup windows and display messages.

Parameter: s: message string that can support Python built-in data types: string, list, boolean, integer, float and other data types;
title: title string that can support Python built-in data types: string, list, boolean, integer, float and other data types;
warning: warning message, boolean type data;
error: Error message, boolean type data;
blocking: whether it is a blocking pop-up window (it must be a non-blocking pop-up window in the function of sec).

Return: None

Example: `popup(s="here I am",title="Popup#1",blocking=True)`

9.4 Character

9.4.1 str_at command

```
str_at(src, index)
```

Function: This command is used to provide direct access to string bytes. This interface returns a string that contains characters in the source string that correspond to the specified index.

Parameter: src: source string, string type data;
index: specifies the position in the string, integer type data.

Return: String type data, the character corresponding to the index number, if the index is not valid, it will throw an exception.

Example: `str_at("Hello",0)`
Return value: 'H'
`str_at("Hello", 1)`
Return value: 'e'
`str_at("Hello", 10)`
Return value: The alarm index is out of range
`str_at("", 0)`
Return value: The given source string is empty

9.4.2 str_cat command

```
str_cat(object1, object2)
```

Function: This command is used to implement the two input data according to certain rules to convert to a string and concatenate. The input of two data can be any Python built-in data type. The floating-point number type is formatted as a 6-digit decimal place and the invalid 0 at the end is removed. The maximum length of the concatenated string is 1023 characters, beyond which a runtime exception is triggered.

Parameter: object1: the first data, string, boolean, integer, float, list type data;
object2: the second data, string, boolean, integer, float, list type data.

Return: String type data, a string after stitching.

Example:

```
str_cat("Hello"," World!")
Return value:"Hello World!"
str_cat("Integer ", 1)
Return value:"Integer 1"
str_cat("", [1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
Return value:"[1, 2, 3, 4, 5, 6]"
str_cat([True, False , True], [1, 0, 1])
Return value:"[True, False , True][1, 0, 1]"
str_cat(str_cat("", str_cat("11","22")), str_cat("Three", "Four"))
Return value:"1122ThreeFour"
```

9.4.3 str_empty command

```
str_empty(str)
```

Function: This command is used to return true when str is empty, false otherwise.

Parameter: str: source string, string type data.

Return: Boolean type data, True if the string is empty, False otherwise.

Example:

```
str_empty("")
Return value:True
str_empty("Hello")
Return value:False
```

9.4.4 str_find command

```
str_find(src, target, start_from=0)
```

Function: This command is used to find the first match of the substring target in src, with a zero index of the string.

Parameter: src: source string, string type data;
target: the substring to search, string type data;
start_from: optional starting position (default is 0), integer type data.

Return: Integer type data, if the target string is found in src, the position of the target string is returned. if the target string is not found in src, -1.

Example:

```
Return value:4  
str_find("Hello World!", "lo")  
Return value:3  
str_find("Hello World!", "o", 5)  
Return value:7  
str_find("abc", "z")  
Return value:-1
```

9.4.5 str_len command

```
str_len(str)
```

Function: This command is used to return the number of bytes in a string.

Parameter: str: source string, string type data.

Return: Integer type data, the number of bytes in the input string.

Example:

```
str_len("Hello")  
Return value:5  
str_len("")  
Return value:0
```

9.4.6 str_sub command

```
str_sub(src, index, len)
```


Function: This command is used to return substrings of src. The result is a substring of src, starting with the bytes specified by index and up to len bytes in length. If the requested substring exceeds the end of the original string (that is, $\text{index} + \text{len} > \text{src length}$), the length of the resulting substring is limited to the size of src. If index or len is not in range, an exception is thrown. The string is zero indexed.

Parameter: src: source string, string type data;
index: integer value, specifying the initial byte in the range [0,src length], integer type data;
len: The length of the substring. If len is not specified, it is a string in the range [index, src length], integer type data (optional).

Return: String type data, src starts at the index and intercepts the part of the len character.

Example:

```
str_sub("0123456789abcdefghij",2,0)
Return value:"" (len is 0)
str_sub("abcde", 2, 50)
Return value:"cde"
str_sub("abcde", -5, 50)
Return value: The alarm index is out of range
str_sub("0123456789abcdefghij", 5, 3)
Return value:"567"
str_sub("0123456789abcdefghij", 10)
Return value:"abcdefghij"
```

9.4.7 to_num command

```
to_num(str)
```

Function: This command is used to convert strings to integer or float type numbers. '.' The decimal point is the key to distinguishing the two, but note that scientific notation defaults to float type. Note that the whitespace characters at the beginning and end of the string ("r", "n", "t" and spaces) will be ignored. The legal strings consist of an optional leading "+" "/" "-" sign followed by the following strings (case-insensitive):

- (1). A decimal number consisting of a decimal number (e.g., 40), with '.' Floating-point numbers (e.g., 10.1, -2.,.3), floating-point numbers expressed in scientific notation (e.g., 10E1, 1.5E4).
- (2). Hexadecimal numbers beginning with '0X' (e.g., 0XABC, 0x4a).
- (3). Infinity represented by "INF" or "INFINITY".
- (4). A meaningless number represented by "NAN".

An exception will occur if the number represented by the string is too large (for example, 1.18973e+4932).

Parameter: str: string to convert, string type data.

Return: Float type data or integer type data, the numeric value represented by the string.

Example:

```
to_num("10")
Return value, :10integer
to_num("0xce110")
Return value, :844048integer
to_num("3.14")
Return value, :3.14float type data
to_num("NAN")
Return value:nan
to_num("123abc")
Return value: Alerts the illegal string input
```

9.4.8 to_str command

```
to_str(val)
```

Function: This command is used to get a string representation of a value;
The result string cannot exceed 1023 characters. Floating-point numbers are formatted as 6 decimal places, and trailing 0s will be removed.

Parameter: val: the value to be converted, float type data or integer type data.

Return: String type data, a string representation of a given value.

Example:

```
to_str(10)
Return value:"10"
to_str(3.123456123456)
Return value:"3.123456"
to_str([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
Return value:"[1, 2, 3, 4, 5, 6]"
to_str([True, False , True])
Return value:"[True, False , True]"
```

9.5 Additional

9.5.1 sleep command

```
sleep(t)
```

Function: This command is used for a period of sleep.

Parameter: t: time [s], float type data.

Return: None

Example: `sleep(0.5)`

9.5.2 set_flag command

```
set_flag(n, b)
```

Function: This command is used to set the value of the internal tag bit of the system, similar to the virtual digital output I/O, which is used to save data information when different programs are running.

Parameter: n: index of the flag, integer type data: [0:31];
b: flag value, boolean type data.

Return: None

Example: `set_flag(1,True)`

9.5.3 get_flag command

```
get_flag(n)
```

Function: This command is used to obtain the value of the internal flag of the system, similar to the virtual digital output I/O, which is used to save data information when different programs are running.

Parameter: n: index of the flag, integer type data: [0:31].

Return: Boolean type data, the value of the internal flag.

Example: `get_flag(0)`
Return value: The value of the internal marker bit, True or False

9.5.4 get_steptime command

```
get_steptime()
```

Function: This command is used to return the duration of the robot's time step in seconds. In every time step, the robot controller receives the measured joint positions and velocity from the robot and sends the desired joint positions and velocity back to the robot. The above process occurs at regular intervals with a predetermined frequency. The interval length is the time step of the robot.

Parameter: None

Return: Float type data, the duration of the robot's step in seconds.

Example: `get_steptime()`
Return value: The duration of the robot's step size

9.5.5 controller_timestamp command

```
controller_timestamp()
```

Function: This command is used to obtain the controller's system timestamp (unit: s).

Parameter: None

Return: Controller's system timestamp.

Example:

```
import time
# Get the controller's timestamp
timestamp = controller_timestamp()
# Convert to local time structure
time_local = time.localtime(timestamp)
# Format the data and output to the teach pendant log
textmsg(time.strftime("%D %H:%M:%S", time_local))
```

9.5.6 rtsi_set_watchdog command

```
rtsi_set_watchdog(variable, frequency, action)
```

Function: This command is used to activate a watchdog which can monitor the set frequency of the RTSI input variable. If the monitored frequency is less than the set frequency, the running program will trigger the actions like none, pause or stop. When the program stops, all monitored variables will be no longer monitored.

Parameter: variable: the variable name to be monitored, string data, e.g. "speed_slider_mask";
frequency: the specified frequency; float type data.
action: the action to be triggered, string data, e.g. "ignore", "pause", "stop".

Return: None

Example: `rtsi_set_watchdog("speed_slider_mask", 10, "pause")`

9.5.7 powerdown command

```
powerdown ()
```

Function: This command is used to stop the robot and disconnect the robot and the controller from power.

Parameter: None

Return: None

Example: `powerdown()`

ALWAYS EASIER THAN BEFORE

- **Contact Us**

Inquiries: contact@elibot.com

Technical Support: support@elibot.com

- **Elite Intelligent Robot Co., Ltd. (Manufacturing Site)**

Changyang Road 259, Bldg. 4, 215123, Suzhou (China)

+86-400-189-9358

- **Beijing Elite Technology Co., Ltd.**

South Ronghua Road 10, Bldg. 5, Rm. 611, 100176, Beijing (China)

- **Shanghai Elite Robots Co., Ltd. (Headquarters)**

Chuanhe Road 55, Bldg. 20, 3F, 201210, Shanghai (China)

- **Shenzhen Elite Robots Co., Ltd.**

Wutong Island, Hangkong road, Bldg. 1A, Ste. 202, 518100, Shenzhen (China)

- **Elite Robots Inc.**

10521 Research Dr., Ste. 104, 37932, Knoxville, TN (USA)

- **Elite Robots Deutschland GmbH**

Münchener Str. 53, 85290, Geisenfeld, Bavaria (Germany)

- **Elite Robot Japan Co., Ltd.**

TOSHIN Hirokoji Honmachi Bldg., 1F, 2-4-3 Sakae, Naka-ku,
460-0008, Nagoya (Japan)

- **Mexico Service Center**

Calzada del pedregal 523, fraccionamiento el pedregal



WeChat Official Account